

Renesas RA8 系列

RA 双核 MCU 缓存入门指南

引言

本文档对 Renesas RA8 双核 MCU 系列进行了概述，重点关注其双核架构和缓存（Cache）存储器子系统，旨在帮助开发者理解 RA8 双核 MCU 中与缓存相关的功能、特性、配置和缓存维护需求。

本文档深入探讨了 RA8 双核 MCU 中指令缓存和数据缓存的关键方面，详细说明了其配置、管理方法，以及在使用 DMA（直接存储器访问）等总线主设备时实现数据一致性和确定性执行的技术。

注意：目前，D-Cache 的使用完全由用户负责。用户需要执行所有必要的 D-Cache 维护操作，或确保数据被放置在预定义的不可缓存（Non-cacheable）区域中。

通过本文档，读者将清楚地了解如何利用 Renesas 灵活软件包（FSP，Flexible Software Package）来配置、管理和优化 RA8 MCU 中的缓存，同时保持数据完整性并确保跨多个内核和硬件组件的确定性行为。

本应用笔记涵盖以下关键主题：

- RA8 双核 MCU 总线与缓存架构概述：理解双核系统和缓存层次结构的组织方式，以及它们如何与共享资源交互。
- 指令缓存和数据缓存的实现与管理：有关启用、配置和维护缓存操作以满足不同应用需求的详细指导。
- 缓存利用的优势：通过实际示例展示指令缓存和数据缓存可带来的性能提升。
- 使用 FSP 进行配置：通过 FSP 框架启用和调整缓存功能的逐步说明。
- 系统优化的最佳实践：确保数据完整性、可预测的执行行为以及跨多个内核和硬件组件的缓存一致性的建议。

所需资源

- 灵活软件包（FSP）v6.3.0
- Renesas e² studio — 版本：2025-12
- Segger J-Link RTT Viewer 8.74a 或更高版本

支持的开发板

- EK-RA8P1

支持的 MCU 系列

截至本文档发布时，支持的 MCU 系列包括：

- RA8P1 系列
- RA8T2 系列
- RA8D2 系列
- RA8M2 系列

注意：本应用笔记及随附的示例工程是基于 RA8P1 双核 MCU 作为参考开发的。然而，由于 RA8T2、RA8D2 和 RA8M2 MCU 共享相同的架构，这些资源也可用于指导这些系列中的缓存使用。

目录

RA 双核 MCU 缓存入门指南	1
引言	1
所需资源	1
1. 引言	5
2. 双核系统架构	5
2.1 双核系统概述	5
2.2 系统互连与总线架构	6
2.3 CM85 和 CM33 内核概述	7
2.4 RA8 双核缓存子系统概述	7
2.5 可缓存性与可共享性	8
3. 缓存架构与实现	9
3.1 RA8 MCU 中的缓存类型与功能	9
3.2 CM85 中的指令缓存与数据缓存 (I/D Cache)	9
3.3 CM33 中的代码缓存与系统缓存 (I/D Cache)	10
3.4 CM33 缓存操作与使用注意事项	12
4. 缓存配置	13
4.1 RA8 双核 FSP 缓存支持概览	13
4.2 使用 BSP 设置	14
4.3 使用 CMSIS API	14
4.4 不可缓存缓冲区放置	16
5. 缓存一致性	16
5.1 缓存管理与一致性	16
5.2 共享内存 (CM85 与 CM33 之间)	17
5.3 外设相关的缓存管理	17
5.4 最佳实践	18
6. 示例工程与参考应用	18
6.1 缓存实现示例代码	18
7. 运行应用工程	20
7.1 导入工程	20
7.2 编译工程	20
7.3 下载、运行与验证工程	22
7.4 验证应用	23
7.5 验证缓存维护应用	26
8. 调试与故障排除	30
9. 后续步骤	30

Renesas RA8 Series	Getting Started with Cache On RA Dual-Core MCUs
10. 参考资料	31
• Renesas FSP 用户手册: https://renesas.github.io/fsp	31
网站与支持	31
修订历史	32
微处理器和微控制器单元产品使用一般注意事项	33
声明	33

1. 引言

缓存（Cache）是一种位于 CPU 内核附近的小容量高速存储器。其主要功能是通过存储和检索频繁使用的数据和指令来减少访问延迟，从而最大程度减少对慢速主存的访问次数。由于缓存比主存快得多，因此高效的缓存利用可以显著提升系统整体性能。

缓存存储器不属于系统内存映射的一部分，因此没有物理地址。其内容由硬件控制逻辑自动管理，而非由程序员直接控制。因为缓存对软件透明，所以运行中的应用通常不直接感知缓存行为。对上下文切换或 DMA 传输等操作，必须通过软件显式执行缓存维护（如清理或无效化），以确保数据一致性。

以下章节概述了双核系统及其缓存架构，涵盖缓存组织结构、操作模式，以及连接在 AXI 总线上的存储器（包括内部闪存/MRAM、SRAM、SDRAM 和外部 Octal-SPI 闪存）相关的性能考量。

2. 双核系统架构

2.1 双核系统概述

Renesas RA8 双核 MCU 系列集成了异构双核架构，旨在为高级嵌入式系统提供高性能、安全性和实时响应能力。

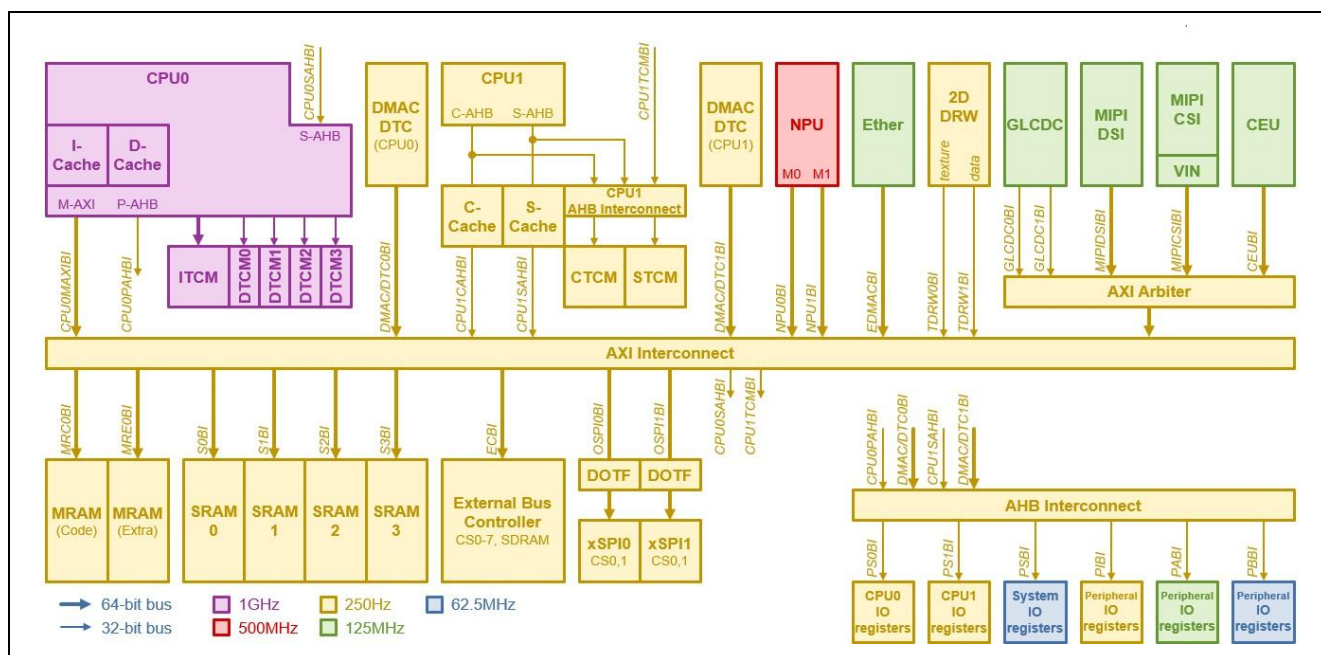
Renesas RA8 双核 MCU 的系统架构经过精心设计，旨在促进内核与所连接外设之间的高效通信和数据交换。该系统主要通过多层次系统总线矩阵（System Bus Matrix）将各个内核和外设连接起来。

为了管理两个内核之间的共享资源访问并协调操作，RA8 双核架构实现了核间通信（IPC，Inter-Processor Communication）机制，包括 FIFO 缓冲区、中断信号、共享内存区域、硬件信号量（Sph）和核间中断（IPI，Inter-Processor Interrupt）。

RA8 双核底层总线架构基于 Arm AMBA（高级微控制器总线架构）标准中的标准高速通信协议，如 AXI、AHB 和 APB。该架构可实现内核、存储器和外设之间高效的数据交换，同时确保关键任务实时处理所需的确定性行为。

RA8P1 双核 MCU 集成了一颗高性能 Arm® Cortex®-M85 内核（最高运行频率 1 GHz），以及一颗 Arm® Cortex®-M33 内核（最高运行频率 250 MHz），并提供以下关键特性：

- 高达 1 MB 的 MRAM，用于非易失性存储器存储
- Cortex®-M85 缓存存储器共 32 KB：
 - o 16 KB 指令缓存（I-Cache）
 - o 16 KB 数据缓存（D-Cache）
- Cortex®-M33 缓存存储器共 32 KB：
 - o 16 KB 指令缓存（C-Cache）
 - o 16 KB 数据缓存（S-Cache）
- 2 MB SRAM，包括：
 - 256 KB TCM RAM（Cortex-M85 内核）
 - 128 KB TCM RAM（Cortex-M33 内核）
 - 1.664 MB 用户可访问 SRAM
- 集成 Arm® Ethos™-U55 神经网络处理单元（NPU）
- 八线串行外设接口（OSPI），用于高速外部闪存访问
- 第 3 层以太网交换模块（ESWM）、USB 全速（USBFS）、USB 高速（USBHS）和 SD/MMC 主机接口
- 图形 LCD 控制器（GLCDC），用于高级显示支持
- 2D 绘图引擎（DRW），用于硬件加速图形渲染
- 支持 MIPI DSI/CSI 接口，用于显示器和摄像头连接
- 丰富多样的模拟外设
- 全面的安全与功能安全特性，确保可靠运行



【图示：图 1. RA8P1 双核系统架构概览】

有关 RA8 双核 MCU 架构的更多信息，请参阅《RA8P1 双核 MCU 开发指南》(R01AN7881) 应用笔记。

2.2 系统互连与总线架构

两个内核通过 RA8 MCU 的系统总线矩阵 (System Bus Matrix) 管理的多层 AXI/AHB 互连结构进行通信并访问共享系统资源。每个主设备 (CM85、CM33、DMA 和外设总线控制器) 都具有独立的端口连接到该总线矩阵，允许并行访问不同的从设备。

互连的关键组件包括：

- **AMBA AXI 总线协议** (高级可扩展接口, Advanced eXtensible Interface): 为 CM85 和 DMA 控制器提供高带宽、低延迟的数据传输。AXI 通道支持突发传输和高效的缓存行填充，用于指令和数据访问。
(此段与上段合并翻译)
- **AMBA AHB 总线协议** (高级高性能总线, Advanced High-performance Bus): 主要用于需要确定性时序的系统外设。
(此段与上段合并翻译)
- **总线主设备:**
 - CM85 内核 (AXI 主设备) ◦ CM33 内核 (AHB 主设备)
 - DMA 控制器
 - 外设 DMA 引擎 (如以太网、USB)
 - 调试访问端口 (DAP)

系统总线矩阵处理这些主设备之间的仲裁和优先级控制，确保来自任一内核的高优先级实时请求得到及时响应，同时维持内存访问的一致性。

2.3 CM85 和 CM33 内核概述

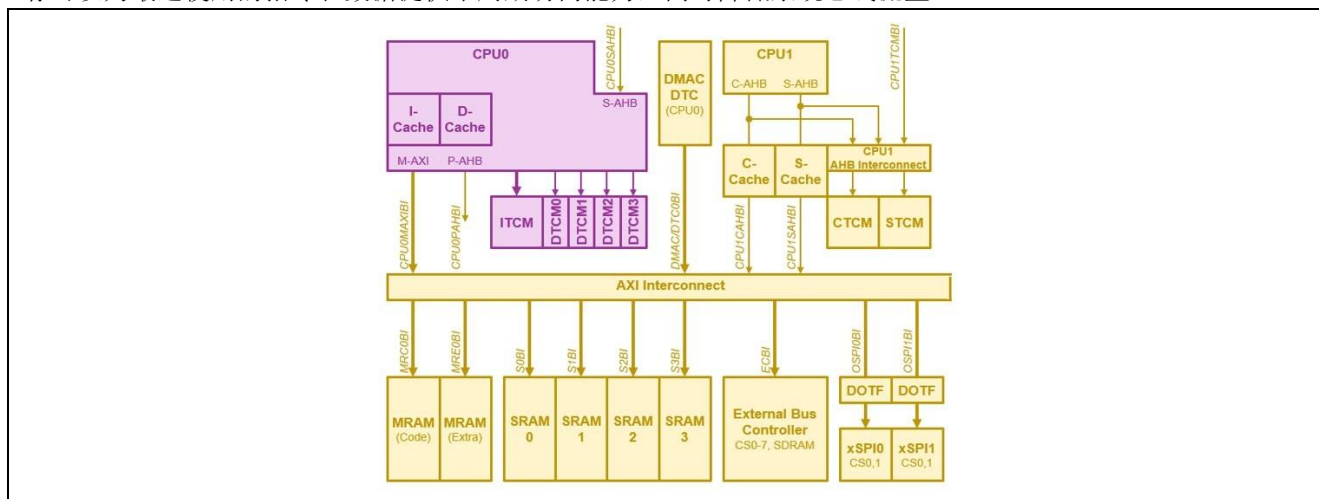
RA8 双核 MCU 将高性能 Cortex-M85 (CM85) 应用核与低功耗 Cortex-M33 (CM33) 管理/安全核配对使用。这种非对称配对设计使得 CM85 可以运行性能关键型应用工作负载，而 CM33 可以处理安全、管理和低功耗任务。

- Cortex-M85 内核 (CM85):
 - Armv8.1-M 架构，支持 Helium (M-Profile 向量扩展，MVE)，用于 DSP 和机器学习应用。
 - 支持单精度 FPU，用于浮点加速。
 - 在最新的 RA8 系列中，CM85 典型工作频率可达 1.0 GHz (早期 RA8M1 型号运行频率较低，如 480 MHz)。这些内核可提供非常高的 CoreMark 性能，适用于计算密集型任务。
 - 缓存：CM85 配备 32 KB 分离式缓存 (16 KB I-Cache + 16 KB D-Cache)，是 RA8 的典型设计，有助于减少外部存储器带宽和延迟，满足实时计算需求。
 - CM85 通常执行用户应用、中间件和性能关键的控制循环。
- Cortex-M33 内核 (CM33):
 - Armv8-M 架构，支持 Arm TrustZone® for M (安全/非安全世界)，非常适合安全启动、密钥管理、加密、外设保护以及运行轻量级 RTOS/监控程序。
 - 运行频率较低 (例如：近期高端 RA8 型号为 250 MHz)，配备 32 KB 缓存 (16 KB 指令缓存 + 16 KB 数据缓存)。
 - 辅助内核可以处理系统管理、安全控制和低功耗操作。CM33 可作为安全监控器，处理安全启动、加密功能或外设保护，通常在运行 RTOS 或裸机监控程序的同时提供服务。

2.4 RA8 双核缓存子系统概述

RA8 缓存是一种位于 CPU 内核附近的小容量高速存储器，用于存储频繁访问的数据和指令，以减少访问慢速主存的次数。

主存访问可能需要多个 CPU 周期，尤其是在外部存储器接口引入等待状态的情况下。为了减少这种延迟，缓存可以为最近使用的指令和数据提供单周期访问能力，同时降低系统总线流量。



【图示：图 2. RA8P1 双核子系统架构概览】

参见图 2 所示的缓存子系统架构，RA8 双核 MCU 为每个 CPU 内核配备了独立的缓存结构。CM85 和 CM33 均具有专用的指令缓存和数据缓存，其操作相互独立。每个缓存专用于其各自的总线主机（CM85 为 AXI 主设备，CM33 为 AHB 主设备）。

对于 Cortex-M85，指令缓存（I-Cache）和数据缓存（D-Cache）都直接集成在内核本身内部。这些是标准 ARM 内核实现的一部分，通过内部寄存器接口进行管理。I-Cache 和 D-Cache 均使用物理地址进行标记，因此不需要进行地址转换。它们对 AXI 总线（CM85 内部）上的访问进行缓存。

而 Cortex-M33 本身不内置缓存功能。Cortex-M33 利用外部缓存单元，具体来说是代码总线缓存（C-Cache）和系统总线缓存（S-Cache）。这些缓存位于 Cortex-M33 内核外部，但集成在 RA8 MCU 内部，通过专用总线接口与 CM33 内核相连。

RA8 缓存被组织为多个缓存行（Cache Line），每行被分组到不同的组（Set）中。每组的行数（即关联性，Associativity）由硬件定义。

当 CPU 发出读取操作时，缓存会判断请求的地址是否已经存储在其中：

- **缓存命中（Cache Hit）：**数据已在缓存中可用，立即返回给 CPU。
- **缓存未命中（Cache Miss）：**数据不在缓存中，必须从主存中获取。获取的数据随后存储在缓存行中以供将来的访问使用。

写操作遵循以下两种策略之一：

- **写透（Write-Through）：**数据同时写入缓存和主存。
- **写回（Write-Back）：**数据仅写入缓存。缓存行被标记为「脏」（Dirty），仅在行被逐出时才写回主存。

如果在未命中时，所选组中的所有缓存行都已占用，缓存会调用其替换算法（通常为伪随机）来选择要逐出的缓存行。如果该行是脏的，则在重新使用之前将其写回内存。

缓存存储器为通过 AXI 总线（CM85）/ AHB 总线（CM33）访问的存储器（包括内部闪存、内部 SRAM 和外部存储设备）提供显著的性能优势。紧密耦合存储器（TCM，Tightly Coupled Memory）不使用缓存，因为 TCM 旨在提供与 CPU 自身速度相匹配的确定性低延迟访问。

支持四种主要的缓存操作：使能、禁用、清理和无效化。Renesas FSP 提供专用的 API 来执行这些操作。CM85 缓存维护使用 CMSIS API，通过直接操作缓存控制寄存器实现缓存操作。

2.5 可缓存性与可共享性

RA8 双核具有一种存储器架构，可以通过 MPU 配置将内存区域配置为可缓存（Cacheable）或不可缓存（Non-cacheable）。为了维护一致性，有时需要将共享内存配置为不可缓存或可共享，以防止数据损坏。

存储器区域	典型用途	可缓存	可访问者
ITCM/DTCM	快速确定性代码/数据（CM85）	否	CM85/CM33
CTCM/STCM	快速确定性代码/数据（CM33）	否	CM33/CM85
SRAM0/1/2/3	共享核间数据和 DMA 缓冲区	可配置	CM85 和 CM33
外部 SDRAM	图形或大型数据缓冲区	是（通过 MPU）	两个内核均可
外部 Flash（OSPI/XIP）	代码存储和 XIP	是（读取）	两个内核均可

两个内核都可以独立访问共享的 SRAM 区域，并且每个内核都可以通过 MPU 和 SAU（用于 TrustZone 的安全属性单元）配置将区域标记为可缓存、不可缓存或可缓冲（Bufferable）。

缓存控制由缓存控制寄存器完成，但 MPU 可以指定缓存策略以及该区域是否可缓存。

缓存控制和 MPU 配置是按内核独立的，CM85 和 CM33 之间不共享。每个内核都有自己的 MPU、I-Cache 和 D-Cache，其缓存使能/禁用、维护操作和 MPU 区域设置完全独立。

如果两个内核需要相同的行为，它们必须各自独立地配置相同的 MPU 区域和缓存策略。

3. 缓存架构与实现

CM85 子系统包含 Arm 实现的 I-Cache 和 D-Cache 单元。CM33 子系统包含 Renesas 实现的 C-Cache 和 S-Cache 单元。

C-Cache 类似于 I-Cache，用于指令读取；S-Cache 用于数据访问。由于 RA8 采用哈佛架构，指令和数据缓存是分开的，以避免取指和数据访问之间的冲突。

在 RA8 MCU 中拥有 I-Cache、D-Cache、C-Cache 和 S-Cache 可显著提升性能和效率。I-Cache 和 C-Cache 通过存储频繁使用的指令来减少指令读取延迟，而 D-Cache 和 S-Cache 通过将近期访问的数据保留在靠近 CPU 的高速存储中来加快数据访问速度。

Cortex-M85 CPU 支持哈佛架构的指令缓存和数据缓存，以提升访问片上或外部存储器时的性能，但只有通过主 AXI 接口的事务才是可缓存的。

在无效化操作期间，缓存操作将被忽略，使用 DSB 指令确保操作完成。软件也可以手动使缓存无效；对于指令缓存，单条指令即可完成，而数据缓存则需要按缓存行进行循环操作。

有关 ARM CM85 缓存实现的更多详细信息，请参考文档——《ARM Cortex-M85 处理器技术参考手册》。

3.1 RA8 MCU 中的缓存类型与功能

指令缓存 (I-Cache):

I-Cache 通过存储最近执行的指令（包括来自内部和外部存储器源的指令），最大限度地减少指令读取延迟。除了提升频繁使用的代码路径的执行性能外，I-Cache 还可以减少总线访问次数并降低功耗。

数据缓存 (D-Cache):

D-Cache 临时存储频繁访问的数据，减少冗余的内存读取并降低系统总线流量。这带来了更快的数据检索和更高的 CPU 效率。此外，D-Cache 支持写回和写透策略，允许根据应用需求灵活配置数据一致性策略。

代码总线缓存 (C-Cache):

此缓存是 Renesas 为 CM33 实现的 I-Cache。C-Cache 通过 C-AHB 接口为 CPU1 的指令访问提供服务，增强外部存储器区域的代码读取性能。

系统总线缓存 (S-Cache):

此缓存是 Renesas 为 CM33 实现的 D-Cache。S-Cache 通过 S-AHB 接口优化 CPU1 的系统级数据访问，有助于减少与外设通信时的延迟，并充当共享系统总线的读取缓冲区。

3.2 CM85 中的指令缓存与数据缓存 (I/D Cache)

3.2.1 I-Cache 与 D-Cache 规格

项 目	I-Cache	D-Cache
容量	16 KB	16 KB
关联性	4 路组相联	4 路组相联
缓存行大小	256 位	256 位
条目数	128 条目/路（4 路组相联）	128 条目/路（4 路组相联）

写策略	写回（含写分配）、写透	写回（含写分配）、写透
替换策略	伪 LRU	伪 LRU
缓存维护	仅无效化	清理、无效化、清理+无效化
启用/禁用	通过 SCB 寄存器	通过 SCB 寄存器
操作对齐	32 字节对齐	32 字节对齐

【图示：图 3. RA8 CM85 I-Cache 与 D-Cache 规格】

3.3 CM33 中的代码缓存与系统缓存（I/D Cache）

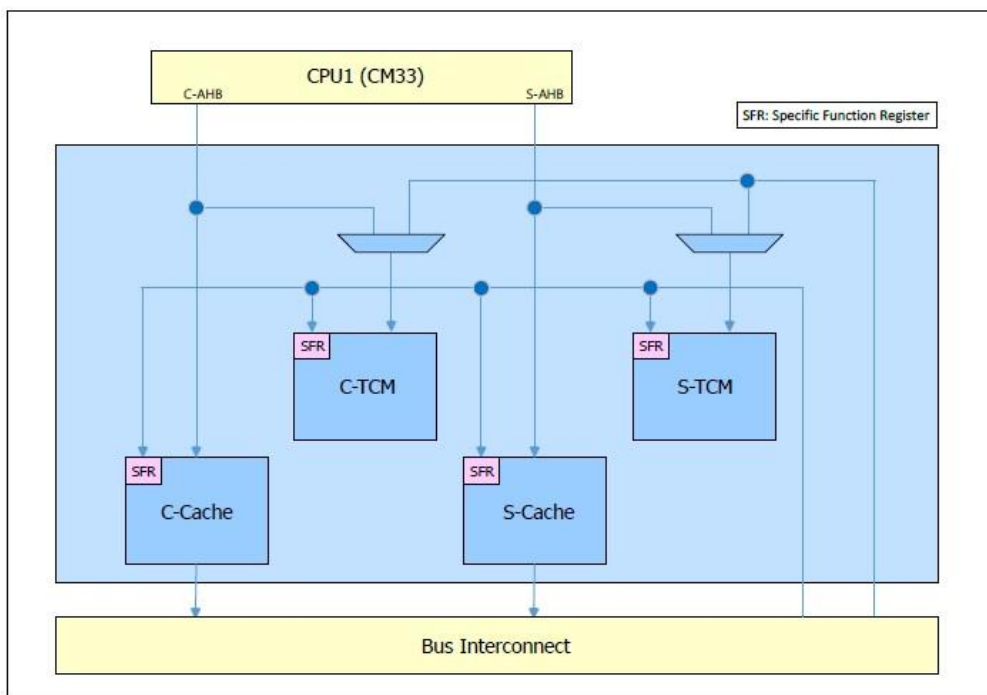
3.3.1 C-Cache and S-Cache Specification

Item	C-Cache	S-Cache
Capacity	16 KB	16 KB
Way	4-way set associative	4-way set associative
Line size	256 bits	256 bits
Number of entry	128 entries/ways (4-way set associative)	128 entries/ways (4-way set associative)
Write way	Write-through/write-back Non write-allocate/write-allocate	Write-through/write-back Non write-allocate/write-allocate
Replace way	LRU	LRU
Error detection	ECC Data memory: SECDED Tag memory: SEDDED	ECC Data memory: SECDED Tag memory: SEDDED
Cache support area	0x0000_0000 – 0x1FFF_FFFF	0x2000_0000 – 0xDFFF_FFFF

【图示：图 4. RA8 CM33 C-Cache 与 S-Cache 规格】

3.3.2 缓存块结构

图 5 展示了 CPU1 的 C-Cache 和 S-Cache 块，以及 C-TCM 和 S-TCM 到总线互连的连接。图 6 展示了 CM33 S-Cache 和 C-Cache 的结构。



【图示：图 5. RA8 CM33 C-Cache 与 S-Cache 块】

Cortex-M33 中的每个缓存行包含几个关键组件：Tag（标记），存储缓存地址位[31:12]；ECC 代码，用于错误检测；以及有效位（V）。V 位表示该缓存行是否包含有效数据。

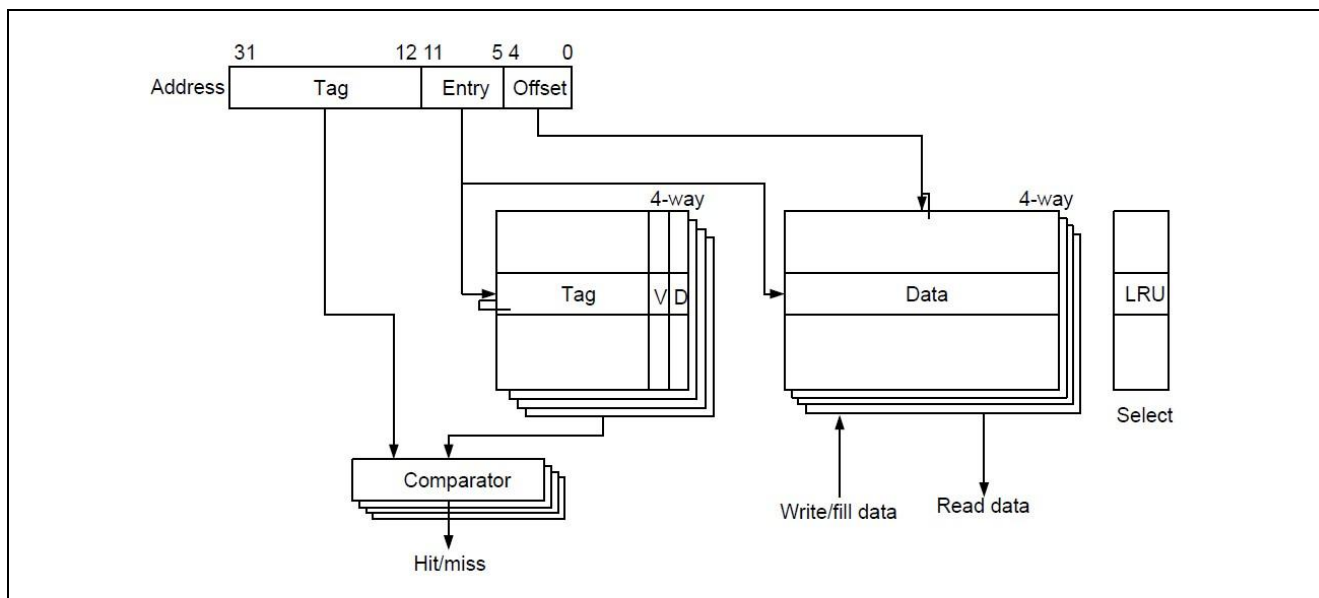


Figure 6. RA8 CM33 C/S-Cache structure

3.4 CM33 缓存操作与使用注意事项

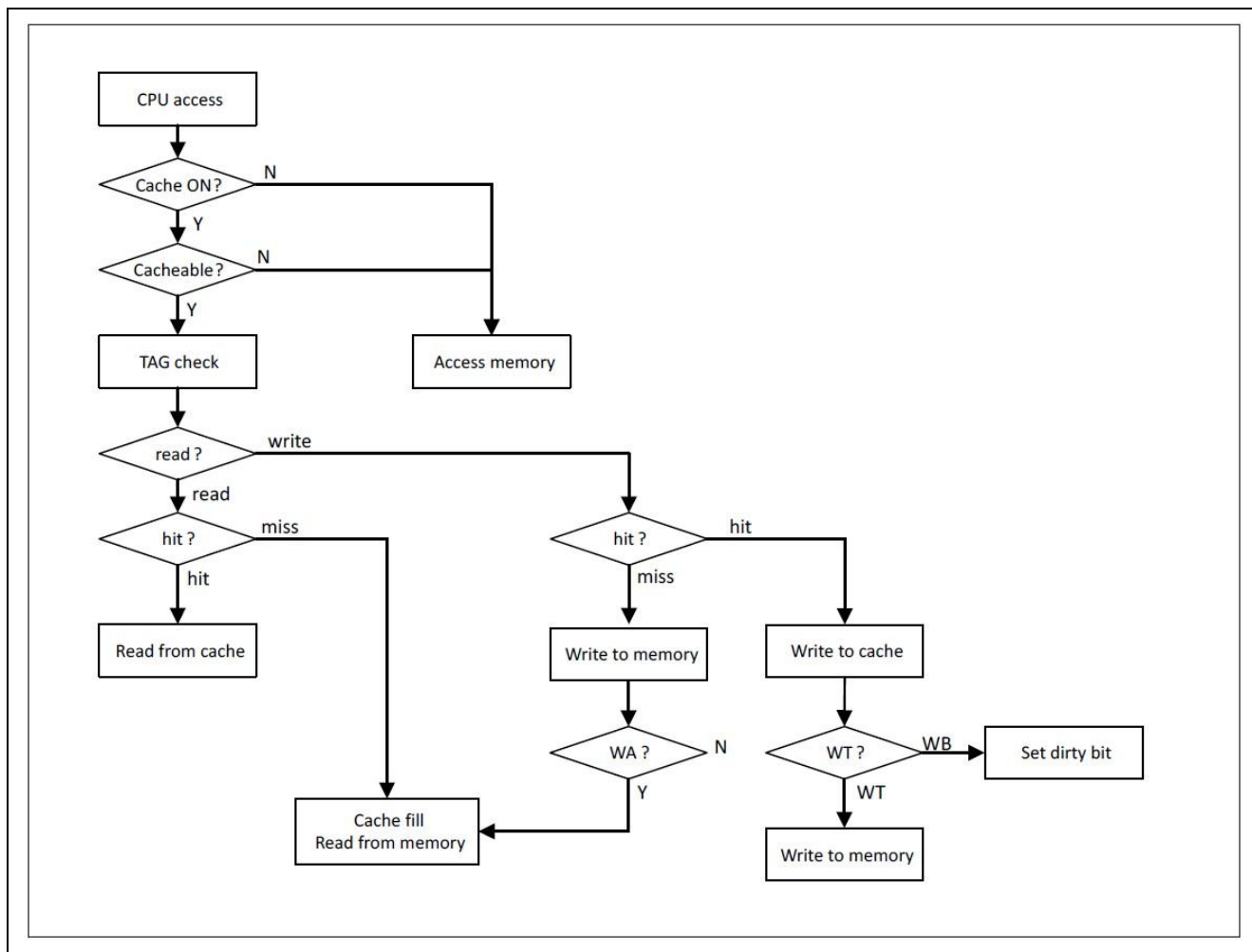


Figure 7. RA8 CM33 C/S-Cache Operation

3.4.1 缓存操作

当缓存处于使能状态（`CCACTL.ENC = 1`，`SCACTL.ENS = 1`）且 CPU 执行可缓存的访问时，缓存会正常工作。它将 CPU 的访问地址与缓存中存储的标记（Tag）进行比较，以判断是否缓存命中。根据比较结果和访问类型，执行如下相应的操作：

- 读未命中（Read miss）：缓存从内存中获取整个缓存行，将其存储在缓存中，然后将请求的数据返回给 CPU。
- 读命中（Read hit）：缓存直接从缓存中提供请求的数据，并以零等待状态完成本次访问。
- 写未命中（Write miss）：缓存将数据写入内存。如果启用了写分配（`CCAWTA.WA = 1`，`SCAWTA.WA = 1`）且访问区域被配置为写分配，缓存还会从内存中加载一个缓存行，然后将数据写入该缓存行。
- 写命中（Write hit）：缓存将数据写入缓存。如果启用了写透模式（`CCAWTA.WT = 1`，`SCAWTA.WT = 1`）或该区域被配置为写透，数据同时也会被写入内存。

3.4.2 缓存刷新

要刷新（Flush）缓存，需将 `CCAFCT.FC` 和 `SCAFCT.FS` 位设置为 1。这将清除有效位、脏位以及标记（Tag）信息。Tag ECC 值将重置为由零生成的 ECC 代码。请注意，刷新操作不会执行写回——它只是清除标记并使所有缓存行无效。

注意：如果后续以不同缓存属性（由 Arm MPU 配置）访问某个地址，则必须在再次访问之前执行写回并刷新（Flush）。当 CCAWTA.WT 和 SCAWTA.WT（写透）位从 1 设置为 0，或者当 CCAWTA.WA 和 SCAWTA.WA（写分配）位从 0 设置为 1 时，也可能发生类似的问题。

在未执行写回和刷新的情况下，以更改后的缓存属性访问相同地址将导致读/写数据行为不确定。

3.4.3 LRU 与替换策略

缓存使用 LRU（最近最少使用，Least Recently Used）策略作为其替换算法。在确定 CPU 访问是命中还是未命中后，缓存通过将访问的行标记为最近最多使用 (MRU) 来更新其 LRU 状态。在未命中时，最近最少使用的行被选择用于逐出。

3.4.4 C-Cache 与 S-Cache 使用注意事项

- 一致性 (Coherency)

软件必须确保缓存与外部存储器以及内部 SRAM 之间的一致性。当在多个总线主设备之间共享内存时（例如，多个 CPU 之间或 CPU 与 DMA 控制器之间），数据一致性是一个关键问题，任何缓存数据都必须与内存保持一致。

- 缓存刷新/写回操作

当 CPU1 使用 *CAFCT/*CACTL 寄存器发起缓存刷新或写回操作时，必须通过确认 CAFCT 返回为 0 来验证操作已完成（* 表示 C-Cache 或 S-Cache 的对应寄存器）。

不要从 CPU1 以外的任何总线主设备触发刷新或写回操作。

在缓存刷新或写回操作期间，缓存将此操作的优先级设置为高于来自 CPU1 的任何访问请求。

- C-Cache 和 S-Cache 安全属性相关的使用限制：在使用 C-Cache 和 S-Cache 时，必须满足以下条件之一：
 - CPU1 必须仅在安全状态 (Secure state) 下运行。
 - CPU1 必须仅在非安全状态 (Non-secure state) 下运行，这仅在 CPU1 的 SECEXT（安全扩展）被禁用时才可能。
 - C-Cache 和 S-Cache 必须仅由非安全程序使用。在这种情况下，还必须满足以下所有要求：
 - 配置 MPU，使安全软件使用的所有安全内存区域标记为不可缓存。
 - 在安全程序访问非安全内存之前，执行缓存写回并刷新。
 - 如果使用片上 WDT 而非外部 WDT，必须为其分配安全属性 (Secure attribute)，并配置为在发生 WDT 错误时中断 CPU0 或触发复位。

4. 缓存配置

4.1 RA8 双核 FSP 缓存支持概览

缓存类型	io-define 中的寄存器定义	维护 API	缓存启用
CM85 I-Cache	已支持	CMSIS API (FSP 内置) FSP	始终启用
CM85 D-Cache	已支持	CMSIS API (FSP 内置) FSP	已支持；可在 BSP 选项卡属性中配置
CM33 C-Cache	已支持	未提供	始终启用
CM33 S-Cache	已支持	未提供	未支持；需在用户代码中实现

注意：尽管 S-Cache 和 C-Cache 的支持在 FSP 中尚不可用，但可以参考本应用笔记随附的示例工程来考虑和开发 C-Cache 和 S-Cache 的类似用法。 [RA Family Guidelines for Using the S Cache on the System Bus \(R11AN0538EU\)](#)”

4.2 使用 BSP 设置

缓存配置可在 e² studio 的 FSP 配置器 BSP 选项卡属性中进行。可以在 e² studio 中通过 BSP -> Cache settings -> Data cache properties 面板为项目配置 D-Cache。此外，I-Cache 始终默认启用。

此外，缓存的 ECC 功能也可以在 e² studio 中通过 BSP 选项卡 -> OFS1 register settings -> Tightly Coupled Memory (TCM)/Cache ECC properties 面板来启用或禁用。

注意：BSP 设置对 CPU0 和 CPU1 均可用。当前，FSP 中尚未提供 CPU1 的 S-Cache 驱动实现。

4.3 使用 CMSIS API

要使用 CM85 缓存，CMSIS 提供了 I-Cache 和 D-Cache 的 API 函数，可根据需要在应用中使用这些 API 来配置和操作缓存。I/D Cache 的 API 及其功能如下表所示。

API 名称	功能	使用说明
SCB_EnableICache	如果 I-Cache 分配尚未启用，首先使整个 ICache 无效化，然后通过 SCB.CCR.IC 位启用 I-Cache 分配 SCB.CCR.IC bit	如果 I-Cache 分配已启用，此函数无操作。FSP 在启动时启用 I-Cache，因此典型应用无需调用此函数。
SCB_DisableICache	通过 SCB.CCR.IC 位禁用 I-Cache 分配，然后使整个 ICache 无效化	此函数确保所有 I-Cache 行被无效化，且不会发生进一步的缓存分配。I-Cache 行永远不会脏（即不包含未写回的数据）。
SCB_InvalidateICache	使整个 ICache 无效化	此函数随时调用都是安全的，因为 I-Cache 行永远不会脏。它丢弃所有缓存的指令，下次取指时将从内存中获取。Flash program or erase mode in the Flash HP driver.
SCB_InvalidateICache_by_Addr	循环使 ICache 中从指定地址开始的指令无效化	此函数随时使用是安全的，因为 I-Cache 中的缓存行永远不会脏。当应用更新内存中的指令后需要刷新特定代码区域时使用。

API 名称	功能	使用说明
--------	----	------

SCB_EnableDCache	如果 D-Cache 分配尚未启用，循环使整个 DCache 无效化，然后通过 SCB.CCR.DC 位启用 D-Cache 分配	如果 D-Cache 分配已启用，此函数无操作。FSP 在启动时会自动调用此函数（如果 BSP 中启用了 D-Cache）。BSP_CFG_DCACHE_ENABLED is defined and non-zero.
SCB_DisableDCache	通过 SCB.CCR.DC 位禁用 D-Cache 分配，然后循环清理并使整个 DCache 无效化 with SCB.CCR.DC bit, then loop to clean and invalidate the entire D-Cache.	禁用系统控制块（SCB）中的数据缓存。建议：D-Cache 不应在运行时频繁禁用/启用，否则会导致性能下降和一致性问题。 Suggestion: The D-Cache may contain dirty lines when you disable it, clean first to avoid losing modified data.
SCB_InvalidateDCache	循环使整个 D-Cache 无效化	此函数通常在系统初始化期间使用；然而，某些场景（如 DMA 传输后）也可能需要调用它。注意：仅执行无效化，不执行清理——脏数据将丢失。
SCB_CleanDCache	循环清理整个 D-Cache	将整个 D-Cache 中所有脏行写回（刷新）到主存。保留缓存行内容但清除脏状态。在将内存共享给其他主设备之前使用。-Cache to main memory. Lines remain valid and stay in the cache.
SCB_CleanInvalidateDCache	循环清理并使整个 D-Cache 无效化	将整个 D-Cache 中所有脏行写回到主存，然后使所有缓存行无效化。这是最全面的缓存维护操作。
SCB_InvalidateDCache_by_Addr	循环使 D-Cache 中从指定地址开始、跨越指定字节数的数据无效化	将以缓存行（32 字节）为增量执行无效化操作。
SCB_CleanDCache_by_Addr	循环清理 D-Cache 中从指定地址开始、跨越指定字节数的数据	将以缓存行（32 字节）为增量执行清理操作。
SCB_CleanInvalidateDCache_by_Addr	循环清理并使 D-Cache 中从指定地址开始、跨越指定字节数的数据无效化 invalidate the data in the DCache, starting at a particular address and extending for the specified length in bytes.	将以缓存行（32 字节）为增量执行清理并无效化操作。

4.4 不可缓存缓冲区放置

当启用 TrustZone 时，预定义的不可缓存段仅在其各自的安全状态内被视为不可缓存。此行为源于 FSP 和启动代码应用的 MPU 配置。如果 TrustZone 被禁用，则整个内存空间都可以配置这些不可缓存属性。

此外，预定义的不可缓存段仅在其各自的内核中被识别。在双核器件上，每个内核都有自己的 MPU，可以独立配置内存区域。因此，CPU0 的不可缓存段在 CPU1 上不一定被视为不可缓存——反之亦然。

当 D-Cache 启用时，由 MPU 配置的预定义不可缓存区域可用于存放不应被缓存的数据，确保这些数据的一致性。要使用预定义的不可缓存段，可以使用以下代码示例将变量或缓冲区放入不可缓存的内存段中。

不同编译器下将缓冲区放入不可缓存区域的示例，包含初始化选项。

（代码块—保留不翻译）

区域	初始化方式	段名称（GCC, IAR, LLVM）	段名称（AC6）
SRAM	未初始化	.ram_noinit_nocache	.bss.ram_noinit_nocache
SRAM	零初始化	.ram_nocache	.bss.ram_nocache
SDRAM	未初始化	.sdram_noinit_nocache	.bss.sdram_noinit_nocache
SDRAM	零初始化	.sdram_nocache	.bss.sdram_nocache
OSPI0 CS0	未初始化	.ospi0_cs0_noinit_nocache	.bss.ospi0_cs0_noinit_nocache
OSPI0 CS0	零初始化	.ospi0_cs0_nocache	.bss.ospi0_cs0_nocache
OSPI1 CS0	未初始化	.ospi1_cs0_noinit_nocache	.bss.ospi1_cs0_noinit_nocache
OSPI1 CS0	零初始化	.ospi1_cs0_nocache	.bss.ospi1_cs0_nocache

5. 缓存一致性

5.1 缓存管理与一致性

在系统中使用任何类型的缓存时，都必须谨慎管理数据一致性（Data Coherency）。缓存可能持有与主存不同的数据，导致数据状态不一致。有效的缓存管理对于确保 CPU、DMA 控制器和其他总线主设备之间的数据完整性至关重要。

通常，一致性可以通过硬件和/或软件机制来维护。在 RA8 系列等 Cortex-M 器件中，没有自动硬件一致性机制（如监听控制单元），因此一致性必须完全通过软件来管理。

每个缓存行支持通过内存保护单元（MPU）属性控制的可配置写回和写透策略。然而，缓存一致性不会在内核之间或内核与总线主设备之间自动维护。当内核或 DMA 控制器通过不同的缓存或直接内存访问修改共享数据时，如果不进行显式的缓存维护，就会出现数据不一致。

默认情况下，FSP 会自动启用 CM85 的 I-Cache 并在必要时管理其一致性。然而，FSP 不会处理其控制范围之外这些缓存的一致性。CM85 的 D-Cache 默认是禁用的——用户可以自行决定是否启用。如果启用了 D-Cache，用户必须承担全部责任，确保软件中正确处理缓存一致性。

当 D-Cache 启用时，最常见的一致性问题发生在 CPU 与另一个总线主设备之间共享数据时。在这种情况下，D-Cache 与对应的主存区域（通常为 SRAM）之间的数据可能不一致。在将数据传递给外设/DMA 以读取共享数据时，CPU 必须将数据从 D-Cache 写回（清理），并在 CPU 读取共享数据之前使 D-Cache 中的缓存数据无效化。

这些操作涉及显式指令来清理（Clean）、无效化（Invalidate）或刷新（Flush）缓存行。当 D-Cache 启用时，为了维护缓存一致性，通常需要显式的缓存维护操作。这些维护操作可以通过 CMSIS 函数完成。

Renesas FSP 管理指令缓存的启用和配置，而本应用工程通过示例软件演示了如何启用、禁用和刷新数据缓存。

5.2 共享内存（CM85 与 CM33 之间）

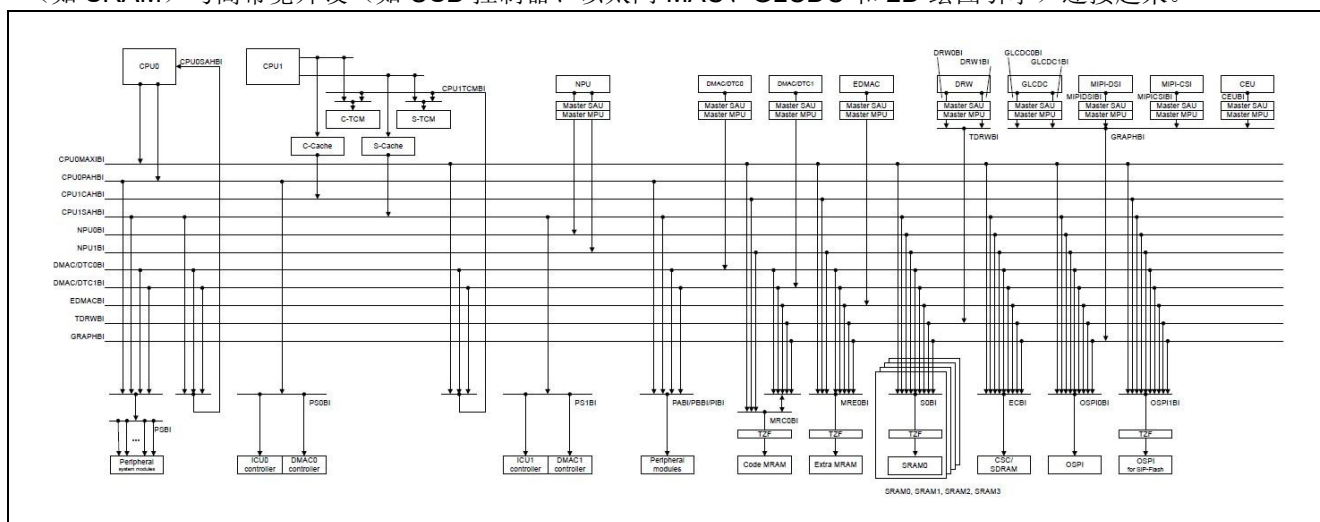
当在 RA8（Cortex-M85 + Cortex-M33）等多核 MCU 上启用 I-Cache 和 D-Cache 时，正确处理共享内存对于避免数据不一致至关重要。每个内核可能会缓存共享内存的独立副本，这意味着其他内核可能无法立即看到任一内核所做的更改——除非通过适当的缓存维护操作显式地使其可见。

另一种方案是，设置共享内存区域并将共享内存标记为不可缓存（如果性能影响可接受的话）。或者，在 MPU 中使用写透缓存或可缓存但可共享的属性，以在保持某些缓存优势的同时减少软件维护开销。

5.3 外设相关的缓存管理

缓存可以提升性能，但当外设（USB、绘图引擎、图形 LCD 控制器、摄像头输入捕获、NPU）直接访问内存时，也可能导致数据一致性问题。CPU 可能已更新了其缓存中的数据，但这些更新可能尚未写回到主存中，从而导致外设读取到过时的数据。

图 8 展示了 RA8P1 的系统总线连接关系。其中，CPU 内核连接到一个中央系统总线，该总线将高速存储器（如 SRAM）与高带宽外设（如 USB 控制器、以太网 MAC、GLCDC 和 2D 绘图引擎）连接起来。



DMA（通用）：对于任何 **DMA** 事务（**ADC**、**SDHI**、**SPI** 等），**DMA** 读取前：清理源缓冲区的缓存。**DMA** 写入后：使目标缓冲区的缓存无效化。

使用 CMSIS API 进行缓存维护：

```
SCB_CleanDCache_by_Addr()、  
SCB_InvalidateDCache_by_Addr()、
```

将缓冲区对齐到缓存行大小（通常为 32 字节）以避免部分缓存行问题。

5.4 最佳实践

正确启用和初始化缓存：

1. 确保在系统初始化早期启用 I-Cache 和 D-Cache，以获得最佳性能。
2. 在启动时使缓存无效化，以避免上次运行残留的过期数据或指令。
1. 优化内存区域：
2. 在 MPU（内存保护单元）设置中正确标记可缓存区域。避免缓存内存映射的外设寄存器或 DMA 访问的区域，以防止一致性问题。
1. 处理缓存一致性：
2. 在使用 DMA 或外设时，在传输前后刷新（清理）或无效化 D-Cache 行。使用 CMSIS 缓存维护 API（例如 SCB_CleanDCache_by_Addr、SCB_InvalidateDCache_by_Addr）进行安全操作。

利用局部性（Locality）：

1. 设计代码和数据结构以利用时间局部性和空间局部性：
2. 将频繁访问的数据放在一起。
3. 使用紧凑循环，避免分散的内存访问模式。

最大限度减少缓存抖动（Cache Thrashing）：

1. 避免使用超过缓存大小的大型工作集。将数据结构对齐到缓存行边界（通常为 32 字节），以减少冲突。
- 2.

修改 MRAM 或闪存中的代码时的维护：如果应用更新了闪存或 MRAM 中的代码，修改后应使 I-Cache 无效化，以确保正确执行新代码。

谨慎使用写回（Write-Back）策略：

D-Cache 通常使用写回和写分配策略；在将数据共享给外设或其他内核之前，务必确保数据已被清理（Clean）。

6. 示例工程与参考应用

为了展示缓存的使用，本应用笔记提供了两个示例工程。在这些工程中，演示了缓存的基本用法、其好处、缓存维护和一致性。

6.1 缓存实现示例代码

随附工程 'ID_Cache_With_MRAM_SRAM_SDRAM.zip' 演示了启用和禁用 I-Cache 和 D-Cache 对系统性能的影响。该工程在不同缓存配置下运行一组固定的测试代码：无缓存、仅 I-Cache、仅 D-Cache，以及两者均启

用，并通过 J-Link RTT Viewer（SEGGER 实时传输）输出性能数据。结果还比较了 MRAM、SRAM 和 SDRAM 作为测试存储器时的性能。

启用缓存带来的性能提升：当 I-Cache 和 D-Cache 均启用时，性能相比缓存禁用时提升约 10 倍。这一显著的性能提升源于对外部存储器的低延迟缓存访问。

随附的另一个工程 'D_Cache_With_MRAM_SRAM_SDRAM.zip' 专注于演示 D-Cache 在 CPU 和 DMA 操作不同场景下的行为。它强调了缓存配置和缓存维护是如何影响 CPU 与 DMA 控制器之间数据一致性的。

用例 1: D-Cache 禁用。

步骤：

1. CPU 将已知数据写入 SRAM 缓冲区。
2. DMA 将一组固定的数据从 MRAM 传输到同一个 SRAM 缓冲区。
3. CPU 从 SRAM 缓冲区读取最新数据。

行为：由于 D-Cache 被禁用，CPU 直接访问 SRAM。因为没有缓存副本，数据一致性得以维持；CPU 始终读取 DMA 写入的最新数据。

用例 2: D-Cache 启用（不执行无效化）。

步骤：

1. CPU 将已知数据写入 SRAM 缓冲区。
2. DMA 将一组固定的数据从 MRAM 传输到同一个 SRAM 缓冲区。
3. CPU 从 SRAM 缓冲区读取数据。

行为：D-Cache 已启用，但未执行缓存无效化。CPU 从其缓存中读取过时数据（旧数据），而非 SRAM 中更新后的数据。数据一致性未被维持，导致错误结果。

用例 3: D-Cache 启用并执行缓存无效化。

步骤：

1. CPU 将已知数据写入 SRAM 缓冲区。
2. DMA 将一组固定的数据从 MRAM 传输到同一个 SRAM 缓冲区。
3. 在 CPU 读取之前，使缓存无效化。

行为：D-Cache 已启用，但无效化操作确保 CPU 从 SRAM 获取最新数据。数据一致性得以维持，CPU 读取到 DMA 写入的正确数据。

用例 4: D-Cache 启用，使用不可缓存的 SRAM 区域。

步骤：

1. 将 SRAM 缓冲区放置在不可缓存的内存区域。
2. CPU 将已知数据写入 SRAM 缓冲区。
3. DMA 将数据从 MRAM 传输到 SRAM 缓冲区。
4. CPU 从 SRAM 缓冲区读取数据。

行为：尽管 D-Cache 在全局范围内启用，但缓冲区位于不可缓存区域。CPU 直接从 SRAM 读取，无需显式无效化即可确保数据一致性得以维持。

关键点：当 DMA 更新内存而 CPU 依赖于缓存数据时，就会出现数据一致性问题。适当的缓存管理技术，如缓存无效化或使用不可缓存区域，对于维护数据完整性是必要的。在 D-Cache 禁用时没有问题——但在实际系统中，如果目标是高性能，这并不总是可行。

这些场景对于 CPU 和 DMA 频繁共享内存的嵌入式系统和 SoC 至关重要。

7. 运行应用工程

7.1 导入工程

1. 启动 e² studio IDE。
2. 在工作区启动器中选择任意工作区。
3. 选择 File > Import。
4. 在 Import 对话框中选择 General > Existing Projects into Workspace。
5. 从下载的文件夹中选择压缩文件 'ID_Cache_With_MRAM_SRAM_SDRAM.zip'。
6. 选择解决方案工程以及各个内核上开发的示例工程（如下所示），然后点击 Finish。

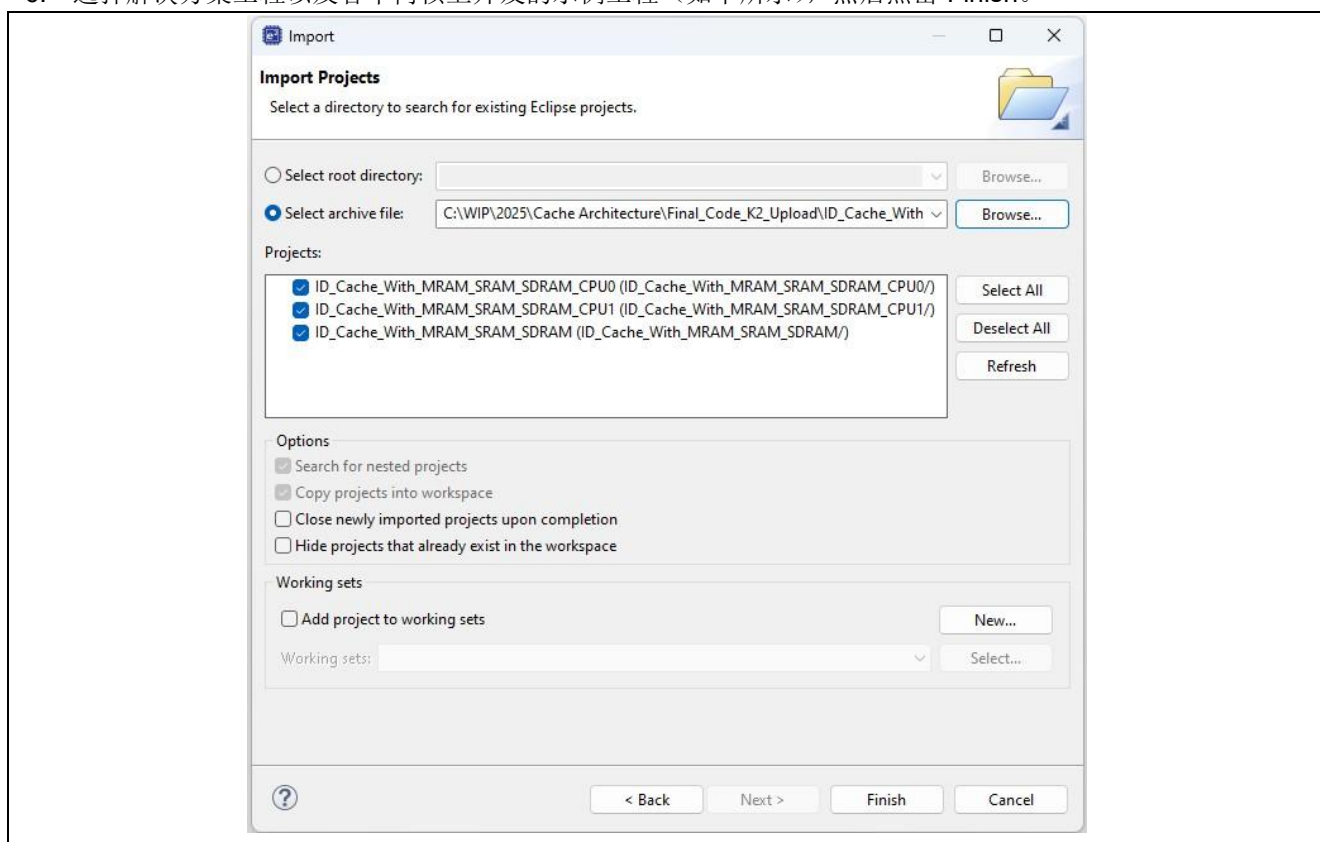
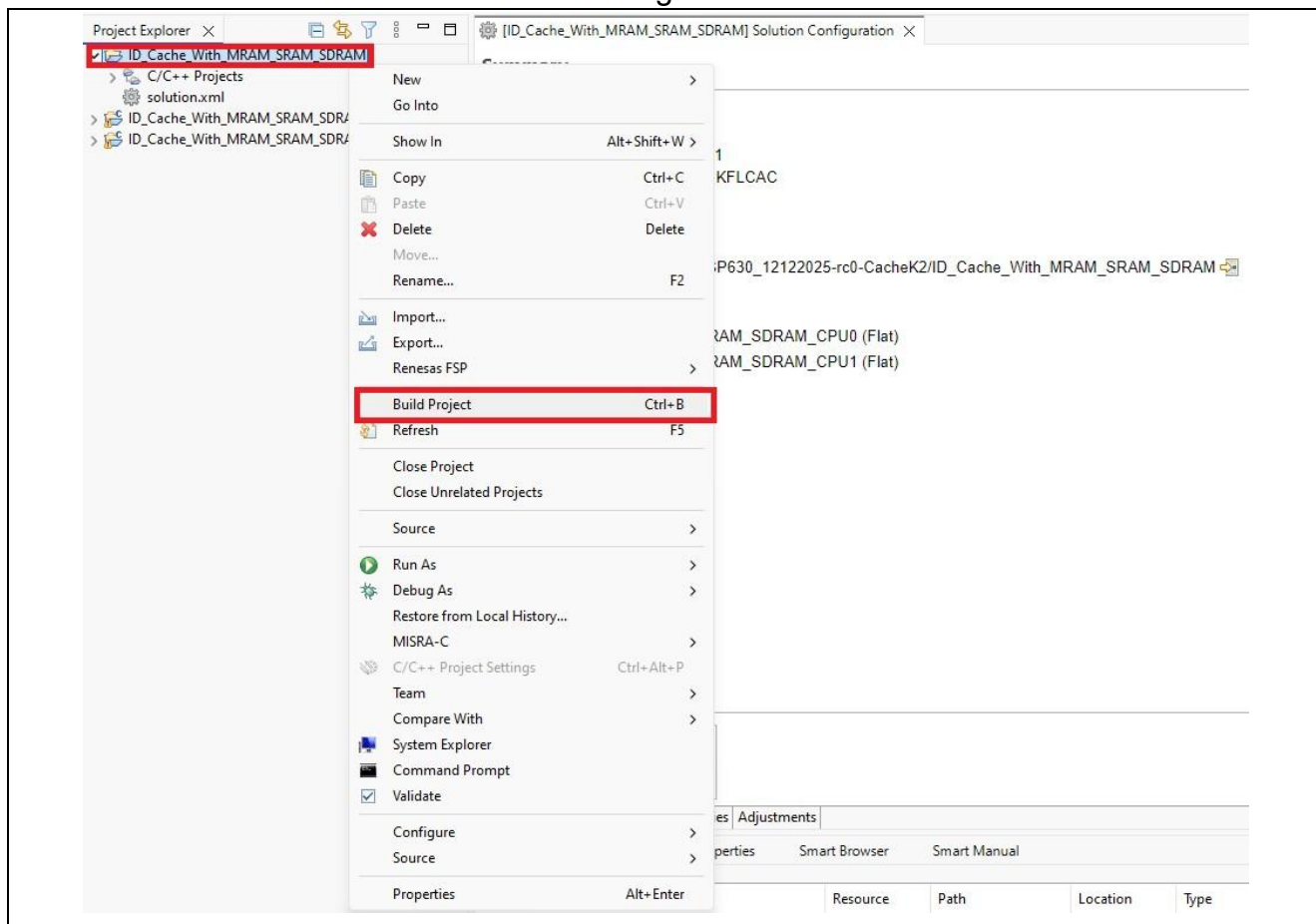


Figure 9. Importing Project.

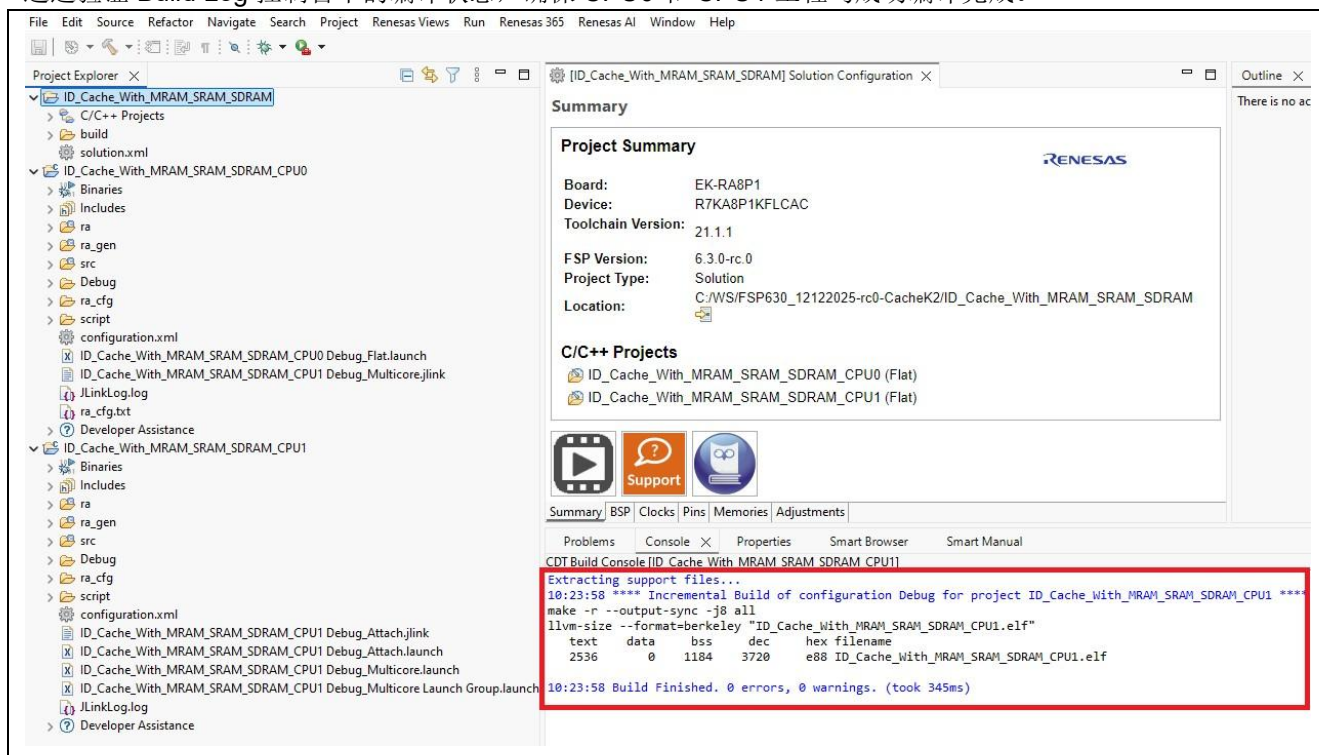
7.2 编译工程

编译导入的解决方案工程 'ID_Cache_With_MRAM_SRAM_SDRAM'，如图 10 所示。这将首先编译 CPU0 工程，然后再编译 CPU1 工程，如图 11 所示。



【图示：图 10. 编译解决方案工程示例】

通过验证 Build Log 控制台中的编译状态，确保 CPU0 和 CPU1 工程均成功编译完成。



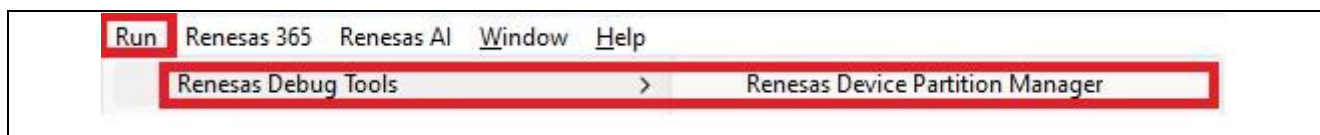
【图示：图 11. 工程编译完成后的编译日志示例】

7.3 下载、运行与验证工程

要同时调试两个内核：

1. 将 USB-C 电缆连接到调试连接器（J10），另一端连接到主机 PC。这将建立 J-Link 连接，用于下载镜像，并通过 Segger J-Link RTT Viewer 启用户交互。
2. 首先，必须将器件初始化为 OEM_PL2 状态，不需要设置 TrustZone 边界。

在 e² studio IDE 中，打开 Renesas Device Partition Manager（Run → Renesas Debug Tools → Renesas Device Partition Manager）。

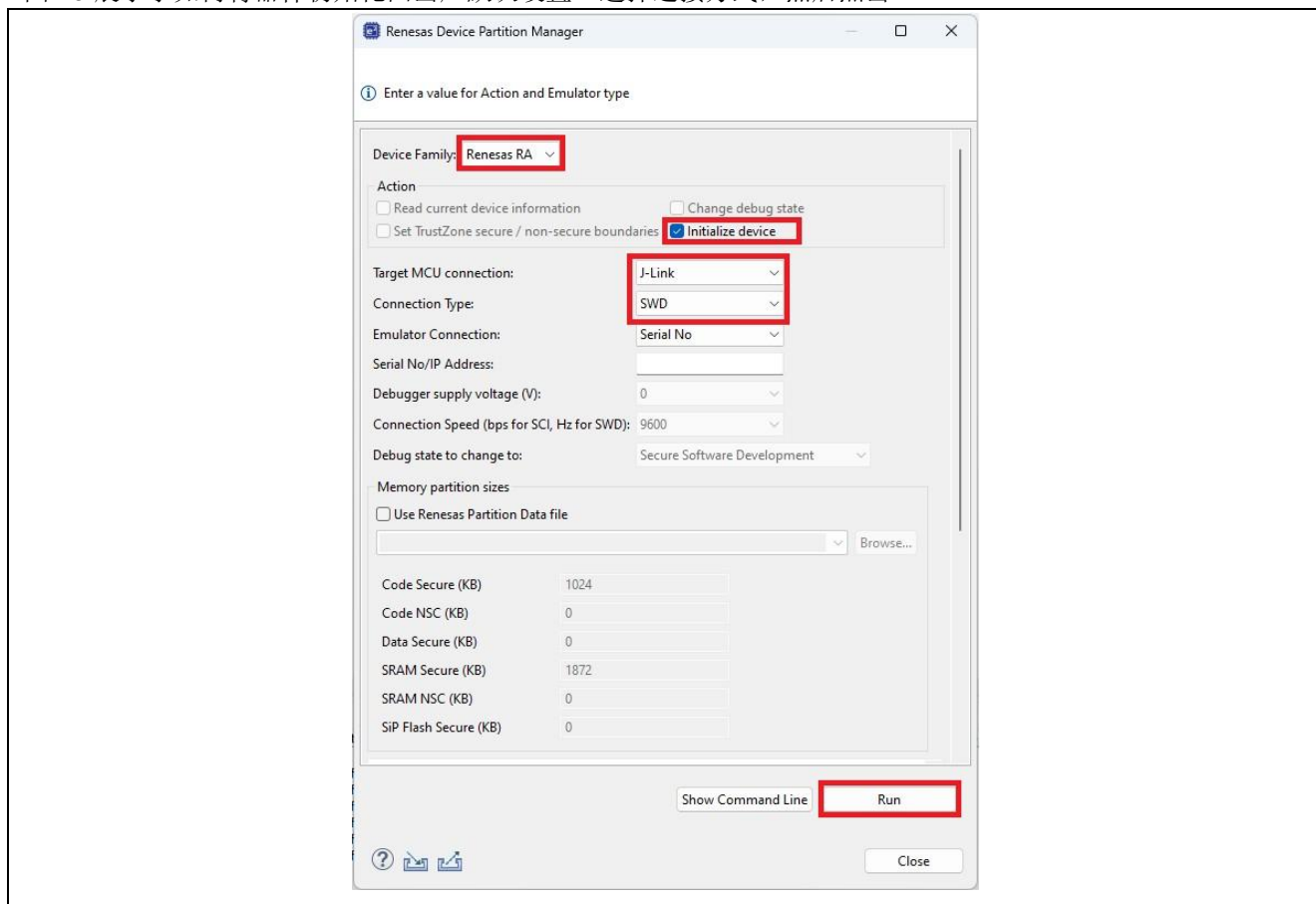


【图示：图 12. 打开 Renesas Device Partition Manager】

注意：要同时调试两个内核，首先使用 Renesas Flash Programmer 或 Renesas Device Partition Manager 初始化 MCU。此步骤确保器件设置为保护级别 2（Protection Level 2），且 TrustZone 边界尚未被配置。如果边界已被配置，可以通过 Renesas Device Partition Manager 将其重置。

完成此初始化是必要的；否则在下载工程镜像或启动调试会话时可能遇到问题。

图 13 展示了如何将器件初始化回出厂默认设置。选择连接方式，然后点击 Run。



【图示：图 13. 使用 Renesas Device Partition Manager 初始化 RA MCU】

3. 打开 Debug Configurations，如图 14 所示。

4. 选择 'ID_Cache_With_MRAM_SRAM_SDRAM_CPU1 Debug_Multicore Launch Group'，如图 15 所示。

5. Click Debug to launch the dual-core debug session.

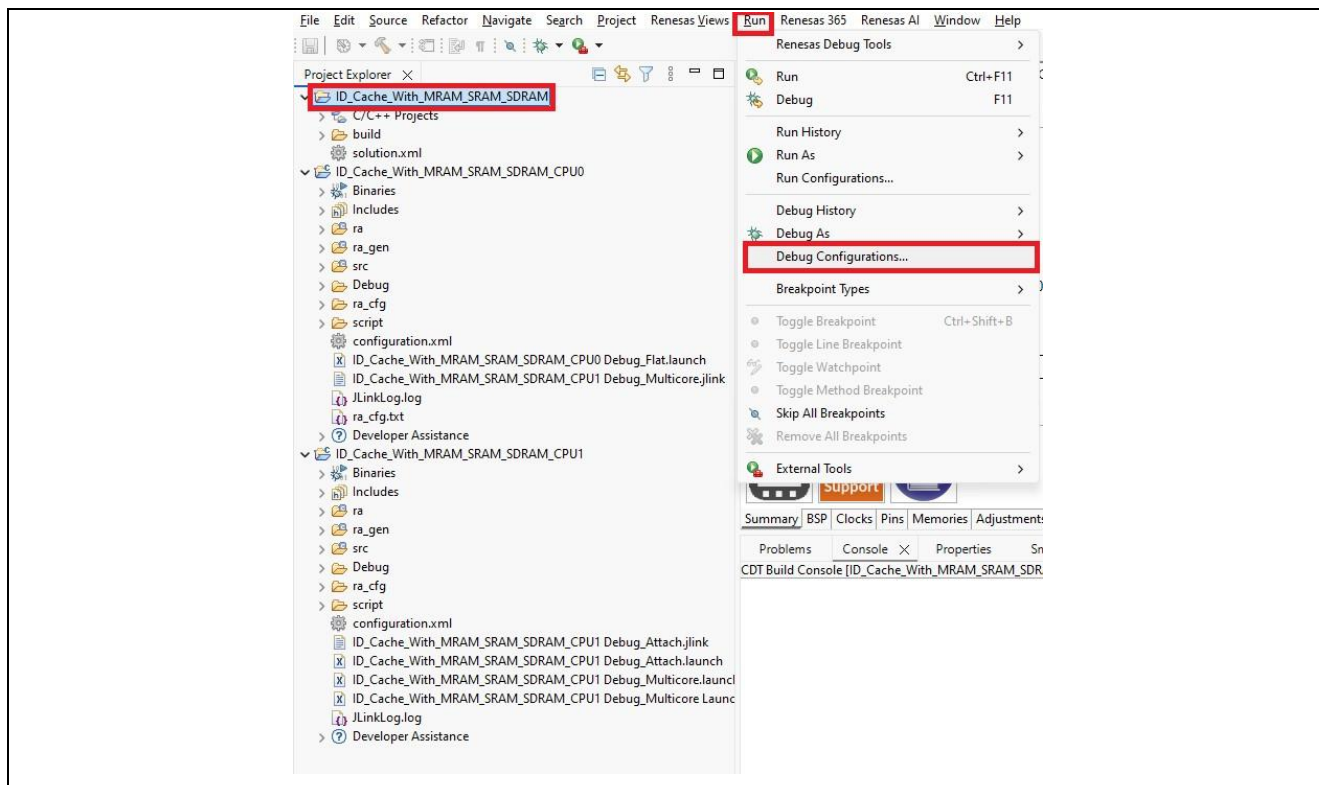


Figure 14. Open Debug Configuration

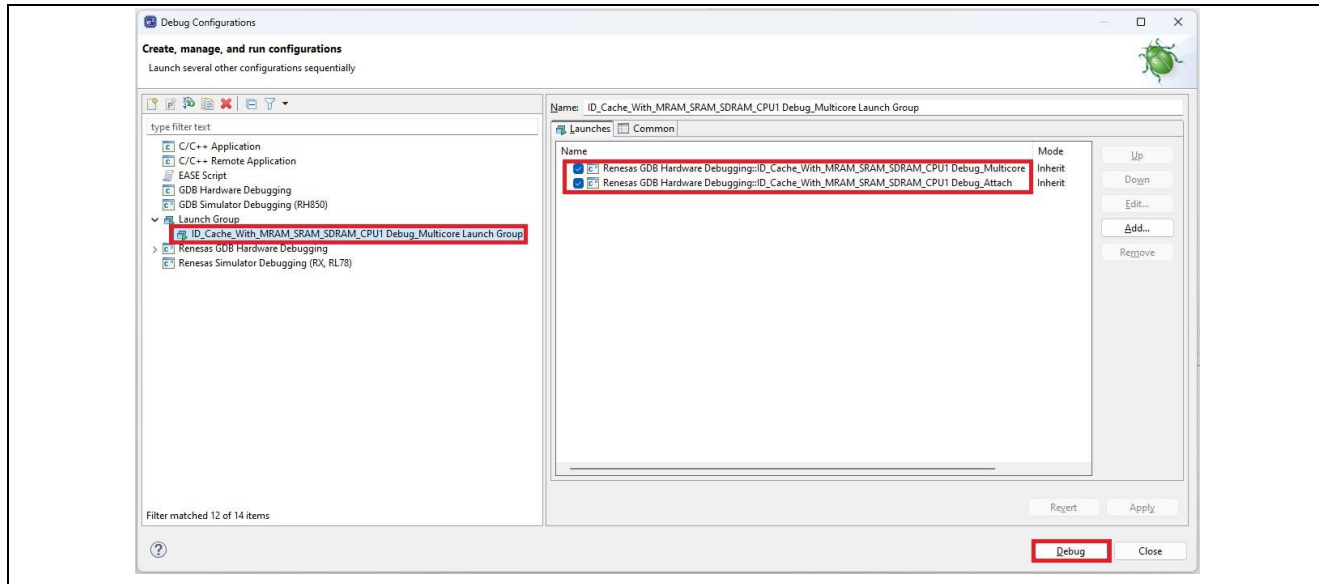
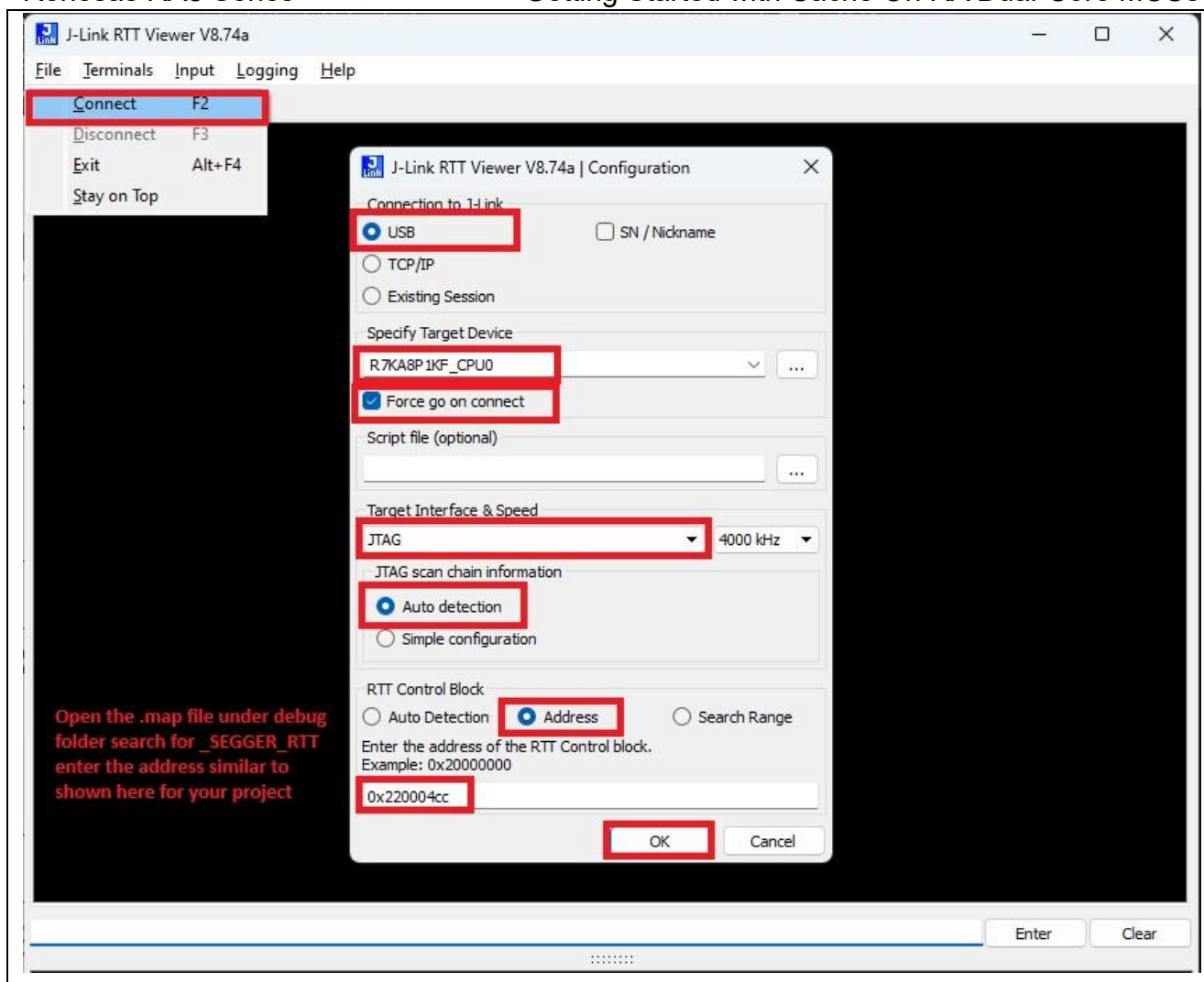


Figure 15. Initiating Debug in the solution project

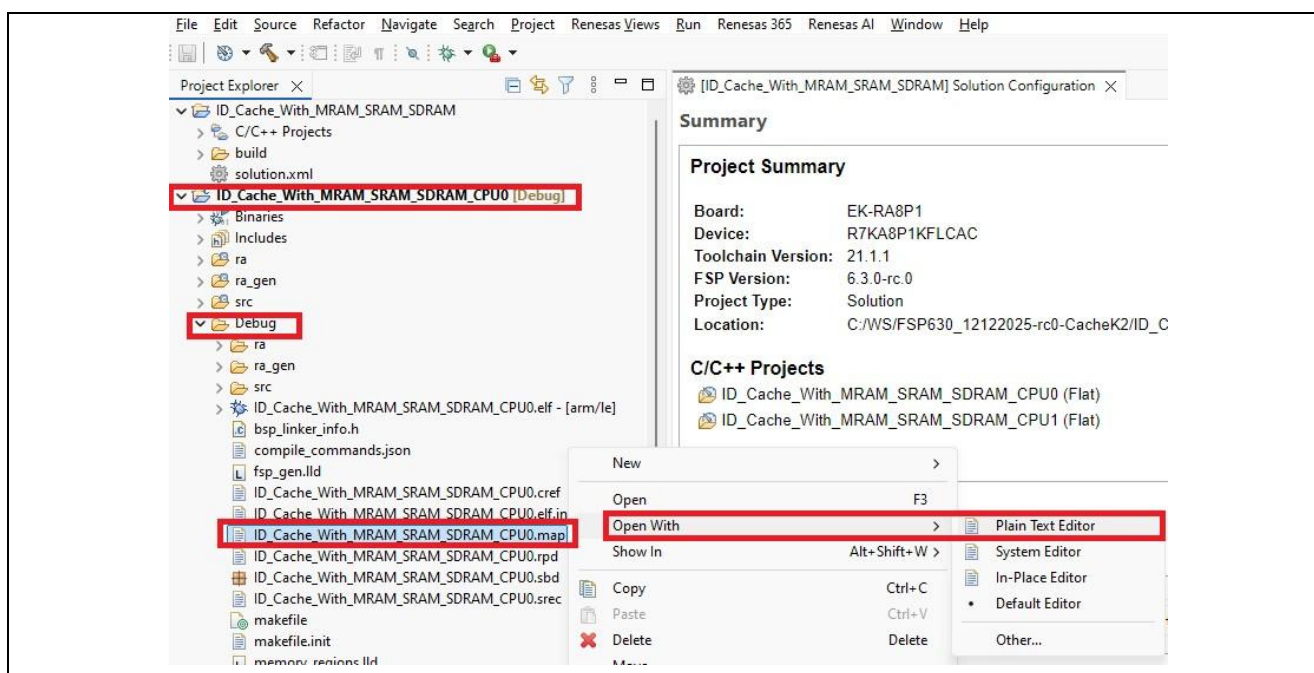
7.4 验证应用

7.4.1 启动并连接 RTT Viewer

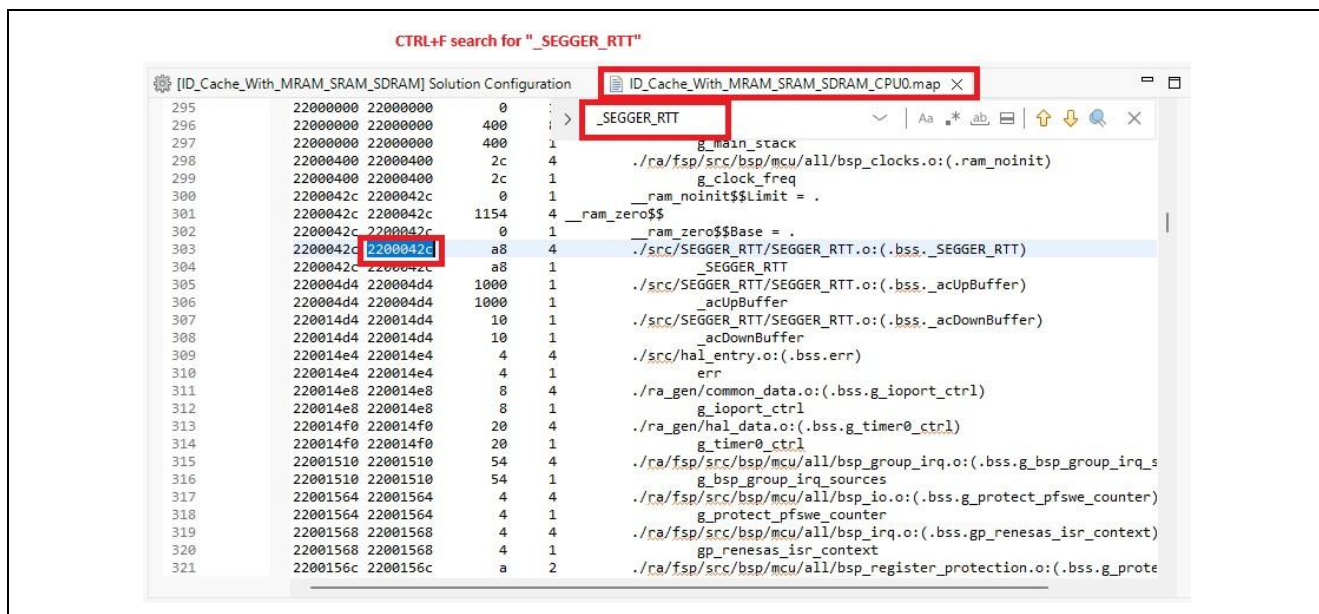
打开 J-Link RTT Viewer (File → Connect)，并使用以下设置进行连接：Connection to J-Link: USB，Specify Target Device: R7KA8P1KF_CPU0（针对 RA8P1），勾选 Force go on connect，Target Interface: SWD，速度为 4000 kHz。



【图示：图 16. 打开 RTT Viewer 并连接的示例】



【图示：图 17. 获取 RTT 控制块地址】



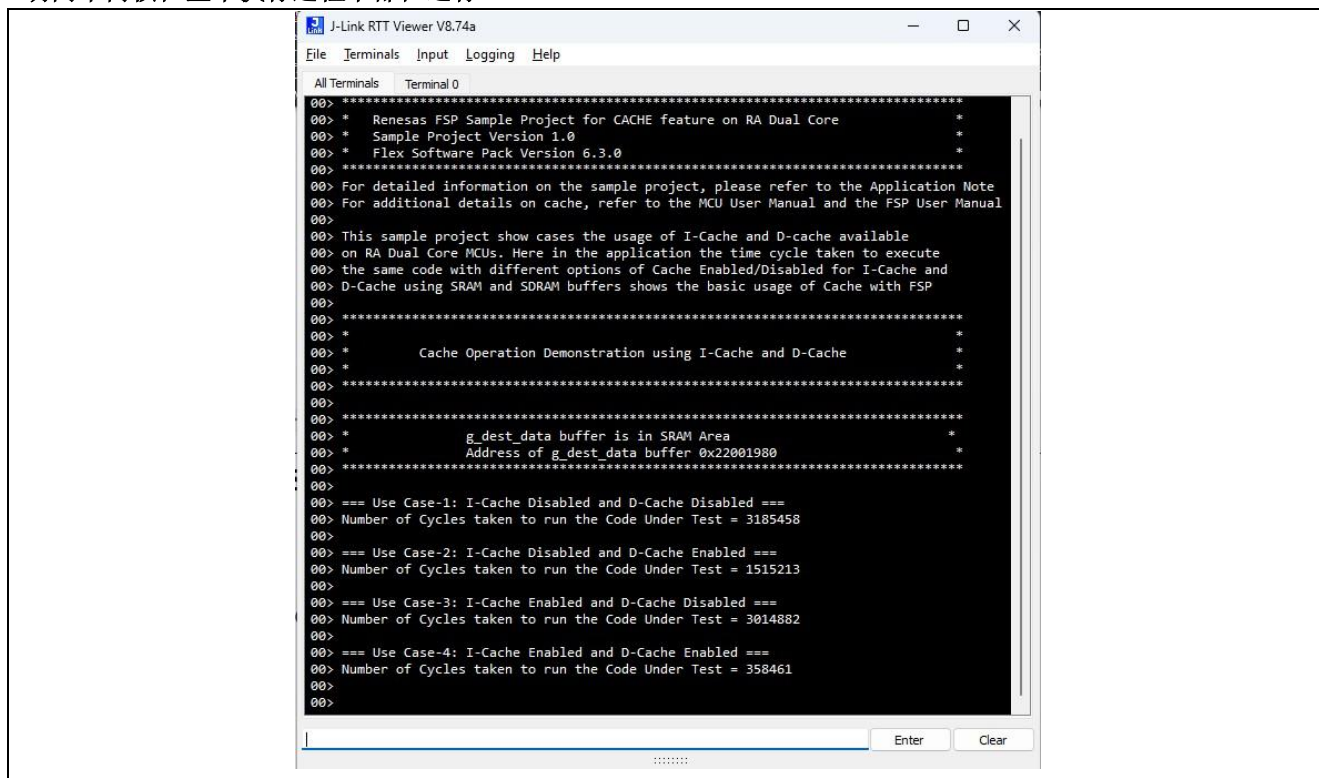
【图示：图 18. 搜索 _SEGGER_RTT 控制块地址】

7.4.1 运行并验证应用

返回 e² studio，按三次 Resume 以在两个内核上执行应用。完成后，示例工程

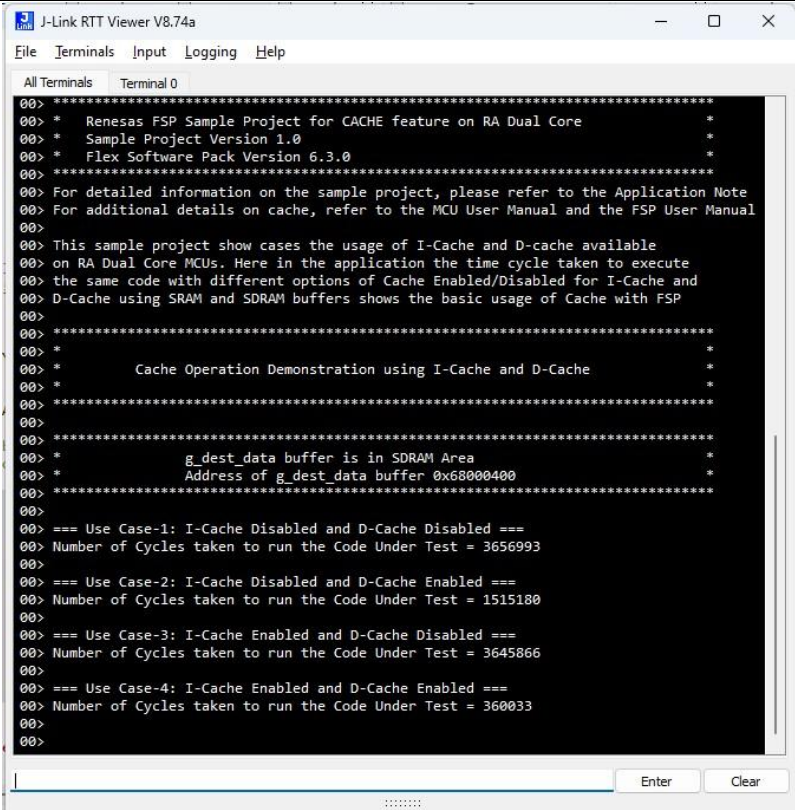
'ID_Cache_With_MRAM_SRAM_SDRAM' 将其状态输出到 RTT Viewer 控制台，显示不同缓存配置在各种存储器类型上的执行时间。

注意：用户可以观察到 LED1（蓝色）和 LED2（绿色）闪烁，因为它们分别由 CPU0 和 CPU1 控制。这也表明两个内核在整个执行过程中都在运行。



【图示：图 19. SRAM 缓冲区示例工程的控制台日志】

注意：示例工程 'ID_Cache_With_MRAM_SRAM_SDRAM' 可以修改为使用 SDRAM 缓冲区运行。为此，打开 'common_utils.h' 并将 `#define MEMORY_REGION SRAM` 修改为 `#define MEMORY_REGION SDRAM`。



```
00> *****
00> *   Renesas FSP Sample Project for CACHE feature on RA Dual Core   *
00> *   Sample Project Version 1.0                                     *
00> *   Flex Software Pack Version 6.3.0                               *
00> *****
00> For detailed information on the sample project, please refer to the Application Note
00> For additional details on cache, refer to the MCU User Manual and the FSP User Manual
00>
00> This sample project show cases the usage of I-Cache and D-cache available
00> on RA Dual Core MCUs. Here in the application the time cycle taken to execute
00> the same code with different options of Cache Enabled/Disabled for I-Cache and
00> D-Cache using SRAM and SDRAM buffers shows the basic usage of Cache with FSP
00>
00> *****
00> *
00> *           Cache Operation Demonstration using I-Cache and D-Cache
00> *
00> *****
00>
00> *****
00> *           g_dest_data buffer is in SDRAM Area
00> *           Address of g_dest_data buffer 0x68000400
00> *****
00>
00> === Use Case-1: I-Cache Disabled and D-Cache Disabled ===
00> Number of Cycles taken to run the Code Under Test = 3656993
00>
00> === Use Case-2: I-Cache Disabled and D-Cache Enabled ===
00> Number of Cycles taken to run the Code Under Test = 1515180
00>
00> === Use Case-3: I-Cache Enabled and D-Cache Disabled ===
00> Number of Cycles taken to run the Code Under Test = 3645866
00>
00> === Use Case-4: I-Cache Enabled and D-Cache Enabled ===
00> Number of Cycles taken to run the Code Under Test = 360033
00>
00>
```

【图示：图 20. SDRAM 缓冲区示例工程的控制台日志】

7.5 验证缓存维护应用

用户可以按照第 7.1 节和 7.2 节中提到的步骤导入和编译第二个工程，并按照第 7.3 节下载和运行 'D_Cache_With_MRAM_SRAM_SDRAM.zip'。

以下各节提供了第二个示例工程的概览，包括其用例和预期输出。

在分析或复现系统行为时，必须明确定义 D-Cache 的行为，因为缓存写策略直接影响内存可见性和测试结果。

强制写透（Force Write-Through）会改变系统行为。在写透模式下，每次存储操作都会立即写入主存，因此数据对于其他代理（如 DMA 引擎）立即可见。这可以掩盖本来由写回引入的一致性问题的。

写回（Write-Back）模式将修改后的数据保留在 D-Cache 中，直到被显式清理或逐出为止，这与实际使用中的性能优化配置相匹配。在写回模式下进行测试有助于暴露仅依赖缓存状态而未进行适当缓存维护时出现的数据一致性问题。

1. 用例 1：D-Cache 禁用

本用例场景：D-Cache 已禁用，缓冲区：g_dest_data 位于 SRAM 地址 0x22000020。

步骤：

1. 初始化：使用 `memset()` 清除缓冲区 → 所有值设置为 0。

2. CPU 更新: CPU 将 AA 写入所有缓冲区位置 → CPU 读取以查看缓冲区是否显示内容为 AA。
3. DMA 传输: DMA 将新的数据模式 (0,1,2,3,4,5,6,7,8,9, A, B, C, D, E, F) 写入同一缓冲区。
4. CPU 读取: DMA 传输完成后, CPU 直接从 SRAM 读取更新后的数据。

关键行为: D-Cache 禁用时, CPU 始终直接访问主存。无需任何缓存维护操作即可维持数据一致性。

- 1.
2. 最终缓冲区内容: 0 1 2 3 4 5 6 7 8 9 A B C D E F

```

All Terminals Terminal 0
00>
00> Type 1: Data Transfer Use Case: D-cache disabled.
00> Type 2: Data Transfer Use Case: D-cache enabled with no cache invalidation.
00> Type 3: Data Transfer Use Case: D-cache enabled and flushed in the application.
00> Type 4: Data Transfer Use Case: D-cache enabled and DMA buffer is in non-cacheable region.
00>
00> < 1
00>
00> *****
00> *
00> *      Cache Operation Demonstration : Use case: D-cache is Disabled      *
00> *
00> * *****
00>
00> *****
00> *
00> *      g_dest_data buffer is in SRAM Area                                *
00> *      Address of g_dest_data buffer 0x22000020                          *
00> * *****
00>
00> Contents of g_dest_data buffer after memset operation.
00> 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
00>
00> CPU is updating the g_dest_data buffer with the data = AA
00> Contents of g_dest_data buffer after CPU writing the data.
00> AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
00>
00> Now different set of data is being written to g_dest_data buffer using DMA operation.
00> New Data being written to g_dest_data.
00> 0 1 2 3 4 5 6 7 8 9 A B C D E F
00> DMAC dma_transfer_operation is in progress.
00> DMAC dma_transfer_operation transfer completed.
00>
00>
00> CPU reads the latest transfered data from the g_dest_data after DMA transfer completion.
00> Data from g_dest_data buffer after DMA transfer.
00> 0 1 2 3 4 5 6 7 8 9 A B C D E F
00>
00> Result: PASS. (The Data is in Sync)
00> Conclusion: As D-cache is Disabled, CPU always access memory directly without the usage of Cache.

```

【图示: 图 21. 缓存行为应用控制台日志: D-Cache 禁用】

2.2. 用例 2: D-Cache 启用 (不执行无效化)

本用例场景: D-Cache 已启用, 未执行缓存刷新。缓冲区: g_dest_data 位于 SRAM 地址 0x22000020。

步骤:

1. 初始化: 使用 memset() 清除缓冲区 → 所有值设置为 0。
2. CPU 更新: CPU 将 AA 写入所有缓冲区位置 → CPU 读取以查看缓冲区是否显示数据为 AA。
3. DMA 传输: DMA 将新的数据模式 (0,1,2,3,4,5,6,7,8,9, A, B, C, D, E, F) 写入同一缓冲区。
4. CPU 读取: CPU 从 D-Cache 中读取过时数据, 而非更新后的内存内容。

关键行为: 在步骤 4 的 CPU 读取之前未使缓存无效化, 因此 CPU 显示的是过时数据。数据一致性未被维持。D-Cache 与 SRAM 之间不同步。

- 1.
2. 结果：DMA 传输后 CPU 读取到错误的结果。
3. Result: Incorrect data read by CPU after DMA transfer.

```
All Terminals Terminal 0
00>
00> Type 1: Data Transfer Use Case: D-cache disabled.
00> Type 2: Data Transfer Use Case: D-cache enabled with no cache invalidation.
00> Type 3: Data Transfer Use Case: D-cache enabled and flushed in the application.
00> Type 4: Data Transfer Use Case: D-cache enabled and DMA buffer is in non-cacheable region.
00>
00> < 2
00>
00> *****
00> *
00> * Cache Operation Demonstration: Use case: D-cache Enabled and No Cache Flush *
00> *
00> *****
00>
00> *****
00> *          g_dest_data buffer is in SRAM Area          *
00> *          Address of g_dest_data buffer 0x22000020      *
00> *****
00>
00> Contents of g_dest_data buffer after memset operation.
00> 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
00>
00> CPU is updating the g_dest_data buffer with the data = AA
00> Contents of g_dest_data buffer after CPU writing the data.
00> AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
00>
00> Now different set of data is being written to g_dest_data buffer using DMA operation.
00> New Data being written to g_dest_data.
00> 0 1 2 3 4 5 6 7 8 9 A B C D E F
00> DMAC dma_transfer_operation is in progress.
00> DMAC dma_transfer_operation transfer completed.
00>
00>
00> CPU prepares to read the latest transfered data from the g_dest_data buffer after DMA transfer completion.
00> CPU reads Stale Data from Cache instead of latest data from g_dest_data buffer after DMA transfer.
00> AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
00>
00> Result: PASS. (The Data is not in Sync as expected)
00> Conclusion: D-cache is Enabled, Cache maintenance is not performed, hence CPU reads stale data from Cache.
00>
```

【图示：图 22. 控制台日志：D-Cache 启用且未执行无效化】

3.3. 用例 3：D-Cache 启用并执行缓存无效化

本用例场景：D-Cache 已启用，使用了缓存刷新（无效化）。缓冲区：g_dest_data 位于 SRAM 地址 0x22000020。

步骤：

1. 初始化：使用 memset() 清除缓冲区 → 所有值设置为 0。
2. CPU 更新：CPU 将 AA 写入所有缓冲区位置 → CPU 读取以查看缓冲区是否显示数据为 AA。
3. DMA 传输：DMA 将新的数据模式（0,1,2,3,4,5,6,7,8,9, A, B, C, D, E, F）写入同一缓冲区。
4. 缓存维护：在 CPU 读取之前执行缓存无效化。
5. CPU 读取：CPU 从内存中获取最新数据，而非过时的缓存数据。

关键行为：缓存无效化确保了 CPU 与 DMA 之间的数据一致性。

1. DMA 传输后最终缓冲区内容：0 1 2 3 4 5 6 7 8 9 A B C D E F。
- 2.

```

All Terminals  Terminal 0
00>
00> Type 1: Data Transfer Use Case: D-cache disabled.
00> Type 2: Data Transfer Use Case: D-cache enabled with no cache invalidation.
00> Type 3: Data Transfer Use Case: D-cache enabled and flushed in the application.
00> Type 4: Data Transfer Use Case: D-cache enabled and DMA buffer is in non-cacheable region.
00>
00> < 3
00> *****
00> *
00> * Cache Operation Demonstration: Use case: D-cache Enabled, Cache Flush used *
00> *
00> *****
00> *****
00> *          g_dest_data buffer is in SRAM Area          *
00> *          Address of g_dest_data buffer 0x22000020      *
00> *****
00>
00> Contents of g_dest_data buffer after memset operation.
00> 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
00>
00> CPU is updating the g_dest_data buffer with the data = AA
00> Contents of g_dest_data buffer after CPU writing the data.
00> AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
00>
00> Now different set of data is being written to g_dest_data buffer using DMA operation.
00> New Data being written to g_dest_data.
00> 0 1 2 3 4 5 6 7 8 9 A B C D E F
00> DMAC dma_transfer_operation is in progress.
00> DMAC dma_transfer_operation transfer completed.
00>
00>
00> CPU prepares to read the latest transfered data from the g_dest_data after DMA transfer completion.
00> CPU reads the latest data from g_dest_data buffer after DMA transfer.
00> 0 1 2 3 4 5 6 7 8 9 A B C D E F
00>
00> Result: PASS. (The Data is in Sync as expected)
00> Conclusion: D-cache is Enabled, Cache maintenance is performed, hence CPU reads latest data
00>          from g_dest_data instead of stale data from Cache.
00>

```

【图示：图 23. 控制台日志：D-Cache 启用且使用了缓存刷新】

4. 用例 4: D-Cache 启用，使用不可缓存的 SRAM 区域

本用例场景中，D-Cache 已启用，但缓冲区（g_dest_data）放置在不可缓存的 SRAM 区域，位于地址 0x22000020。

步骤：

1. 初始化：使用 memset() 清除缓冲区 → 所有值设置为 0。
2. CPU 更新：CPU 将 AA 写入所有缓冲区位置 → CPU 读取以查看缓冲区是否显示数据为 AA。
3. DMA 传输：DMA 将新的数据模式（0,1,2,3,4,5,6,7,8,9, A, B, C, D, E, F）写入同一缓冲区。
4. CPU 读取：CPU 在 DMA 传输后读取更新后的数据。

关键行为：因为缓冲区是不可缓存的，CPU 访问缓冲区内存时不应用缓存。数据一致性得以维持：CPU 读取到 DMA 写入的最新数据。

1. DMA 传输后最终缓冲区内容：0 1 2 3 4 5 6 7 8 9 A B C D E F

```

All Terminals  Terminal 0
00> Type 1: Data Transfer Use Case: D-cache disabled.
00> Type 2: Data Transfer Use Case: D-cache enabled with no cache invalidation.
00> Type 3: Data Transfer Use Case: D-cache enabled and flushed in the application.
00> Type 4: Data Transfer Use Case: D-cache enabled and DMA buffer is in non-cacheable region.
00>
00> < 4
00>
00> *****
00> *
00> * Cache Operation Demonstration: Use case: D-cache Enabled, and the buffer is *
00> * in the non cacheable area *
00> *
00> *****
00>
00> *****
00> *
00> * g_dest_data buffer is in SRAM Area *
00> * Address of g_dest_data buffer 0x22000020 *
00> *****
00>
00> Contents of g_dest_data buffer after memset operation.
00> 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
00>
00> CPU is updating the g_dest_data buffer with the data = AA
00> Contents of g_dest_data buffer after CPU writing the data.
00> AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
00>
00> Now different set of Data is being written to g_dest_data buffer using DMA operation.
00> New Data being written to g_dest_data.
00> 0 1 2 3 4 5 6 7 8 9 A B C D E F
00> DMAC dma_transfer_operation is in progress.
00> DMAC dma_transfer_operation transfer completed.
00>
00>
00> CPU prepares to read the latest transfered data from the g_dest_data after DMA transfer completion.
00> CPU reads the latest data from g_dest_data buffer after DMA transfer.
00> 0 1 2 3 4 5 6 7 8 9 A B C D E F
00>
00> Result: PASS. (The Data is in Sync as expected)
00> Conclusion: D-cache is Enabled, but Data buffer is the non cacheable area, hence CPU reads latest data
00> from g_dest_data, as the the buffer memory cannot be cached.
00>

```

【图示：图 24. 控制台日志：D-Cache 已启用且缓冲区为不可缓存】

注意：示例工程 'D_Cache_With_MRAM_SRAM_SDRAM' 可以修改为使用 SDRAM 缓冲区运行。为此，打开 'common_utils.h' 并将 #define MEMORY_REGION SRAM 修改为 #define MEMORY_REGION SDRAM。

8. 调试与故障排除

- 要同时调试两个内核，首先需要使用 Renesas Flash Programmer 或 Renesas Device Partition Manager 初始化 MCU。此步骤确保器件被设置为 Protection Level 2，并且 TrustZone 边界尚未被配置。如果边界已被配置，可以通过 Renesas Device Partition Manager 将其重置。

完成此初始化是必要的；否则，在下载工程镜像或启动调试会话时可能会遇到问题。

- 此外，使用 Launch Group（启动组），将各个启动配置组合在一起，有助于运行和调试双核工程。
- 另外，确保从 CPU0 工程中调用 'R_BSP_SecondaryCoreStart()' 来运行 CPU1 内核。为了调试目的，可以添加一个断点以确保此代码被调用，以确认 CPU1 内核已启动。

9. 后续步骤

- 要了解有关 EK-RA8P1 套件的更多信息，请参阅 EK-RA8P1 用户手册和设计包，这些资料可在 EK-RA8P1 网页 (renesas.com/ek-ra8p1) 的 Documents 和 Downloads 选项卡中分别获取。renesas.com/ek-ra8p1。

- 要了解如何从零开始创建新的 e² studio 工程，请参阅 FSP 用户手册 (renesas.com/ra/fsp) 中的第 2 章「开始开发」。要了解如何使用 e² studio，请参阅 e² studio 用户手册，该手册可在 e² studio 帮助菜单中找到。
- Renesas 提供了多个示例工程，展示了 RA MCU 的不同能力，这些示例工程可以作为用户开发自定义应用的良好起点。

10. 参考资料

与双核存储器架构和系统设计相关的重要参考资料、数据手册和文档合集。

可参考以下文档以更详细地了解 RA8P1:

- RA8P1 系列用户手册：硬件，文档编号 R01UH1064EJ
- Renesas FSP 用户手册：<https://renesas.github.io/fsp><https://renesas.github.io/fsp>
- • RA8 存储器架构、配置和拓扑入门指南，文档编号 R01AN7880EU
- • Arm Cortex-M85 处理器技术参考手册，文档编号 101924_0002_05_en (<https://developer.arm.com/documentation/101924/0101>)
- • RA8P1 多核设置及双核 Hello World 运行指南，文档编号 R01AN7982EU
- • RA8P1 双核 MCU 开发指南，文档编号 R01AN7881EU
- • RA8P1 MCU 快速设计指南，文档编号 R01AN7883EU
- • Arm Cortex®-M85 处理器技术参考手册，文档编号 101924，可从 Arm 获取。
- • 有关 Cortex-M85 缓存的更多详细信息，请参阅 FSP 文档部分：RA 灵活软件包文档：Cortex-M85 Caches。 [RA Flexible Software Package Documentation: Cortex-M85 Caches](#).
- • 有关 S-Cache 和 C-Cache 的使用，可参考以下应用笔记：RA 系列系统总线 S Cache 使用指南 (R11AN0538EU0130) [RA Family Guidelines for Using the S Cache on the System Bus \(R11AN0538EU0130\)](#)

网站与支持

访问以下网址了解 RA 系列的关键元素、下载组件及相关文档，并获取支持。

RA 产品信息 renesas.com/ra

RA 产品支持论坛 renesas.com/ra/forum

RA 灵活软件包 (FSP) renesas.com/FSP

Renesas 支持 renesas.com/support

版本	日期	说明	
		页码	摘要
1.00	2026.01.02	-	初始版本
		-	

微处理器和微控制器单元产品使用一般注意事项

以下使用注意事项适用于 **Renesas** 的所有微处理器单元和微控制器单元产品。有关本文档所涵盖产品的详细使用注意事项，请参阅相关手册中的具体说明。这些注意事项包括但不限于静电放电、上电处理、未使用引脚、时钟信号和保留地址等。

1. 防静电放电（ESD）注意事项：**CMOS** 器件暴露在强电场中时，可能导致栅氧化层损坏并最终使器件性能下降。必须采取措施防止静电产生，并在处理半导体器件的任何过程中保持适当的防护。
2. 上电处理：上电时产品的状态是不确定的。**LSI** 内部电路的状态不确定，上电时寄存器设置和引脚的状态也是不确定的。务必通过复位操作将内部电路稳定到已知状态。
3. 断电状态下输入信号：器件断电时，不要输入信号或 I/O 上拉电源。由此产生的电流注入可能导致故障，并对器件造成异常损坏。
4. 未使用引脚的处理：按照手册中关于未使用引脚处理的指导方向处理未使用引脚。**CMOS** 产品的输入引脚通常处于高阻抗状态。在未连接引脚的情况下运行可能导致因噪声产生不稳定的操作。
5. 时钟信号：施加复位后，仅在运行时钟信号稳定后才释放复位线。在程序执行期间切换时钟信号时，等待目标时钟信号稳定后再进行切换。不稳定的时钟可能导致故障。
6. 输入引脚电压施加波形：输入噪声或反射波引起的波形失真可能导致故障。例如，如果 **CMOS** 器件的输入由于噪声而保持在 **VIL**（最大值）和 **VIH**（最小值）之间的区域，器件可能发生故障。采取措施防止噪声和反射。
7. 禁止访问保留地址：禁止访问保留地址。保留地址是为未来可能的功能扩展而提供的。请勿访问这些地址，因为 **LSI** 的正确操作无法保证。

在从一种产品更换为另一种产品之前（例如更换为不同型号的产品），请确认此更改不会导致问题。微处理器单元或微控制器单元的特性可能因型号而异，即使它们属于同一系列也是如此。

声明

1. 本文档中的电路、软件和其他相关信息的描述仅用于说明半导体产品的操作和应用示例。您对将这些电路、软件和信息纳入您的产品或系统设计负有全部责任。**Renesas Electronics** 不对因您使用这些电路、软件或信息而导致的任何损失或损害承担任何责任。
2. **Renesas Electronics** 特此明确声明，不对因使用本文档中的信息而导致的侵犯第三方专利权、版权或其他知识产权的行为承担任何担保或责任。
3. 本文档不授予 **Renesas Electronics** 或任何第三方的任何专利、版权或其他知识产权的任何许可，无论是明示、暗示还是以其他方式。
4. 您应负责确定需要从任何第三方获得哪些许可，并获取这些许可以便合法地进口、出口、制造、销售、使用、分销或以其他方式处置任何采用 **Renesas Electronics** 产品或服务的产品。
5. 您不得对任何 **Renesas Electronics** 产品进行全部或部分更改、修改、复制或反向工程。**Renesas Electronics** 对因您或任何第三方进行此类更改、修改、复制或反向工程而导致的任何损失或损害不承担任何责任。
6. **Renesas Electronics** 产品按以下两个质量等级分类：'Standard' 和 'High Quality'。每个 **Renesas Electronics** 产品的预期应用取决于该产品的质量等级：
'Standard': 计算机；办公设备；通信设备；测试与测量设备；音视频设备；家用电器；机床；个人电子设备；工业机器人等。
'High Quality': 交通设备（汽车、火车、船舶等）；交通控制（交通信号灯）；大型通信设备；关键金融终端系统；安全控制设备等。
除非 **Renesas Electronics** 的数据手册或其他 **Renesas Electronics** 文档中明确指定为高可靠性产品或适用于恶劣环境的产品，否则 **Renesas Electronics** 产品不适用于需要极高可靠性或涉及直接生命安全的设备或系统，如航空航天、核控制、医疗生命维持等。
7. 没有半导体产品是绝对安全的。尽管 **Renesas Electronics** 硬件或软件产品中可能实现了任何安全措施或功能，**Renesas Electronics** 对因任何漏洞问题而产生的任何责任概不负责。
RENESAS ELECTRONICS 不保证 **RENESAS ELECTRONICS** 产品或任何使用 **RENESAS ELECTRONICS** 产品创建的系统将不会受到破坏、攻击、病毒、干扰、黑客攻击、数据丢失或盗窃或其他安全入侵（“漏洞问题”）的影响。**RENESAS ELECTRONICS** 对因任何漏洞问题引起或与之相关的任何责任概不负责。

此外，在适用法律允许的范围内，**RENESAS ELECTRONICS** 对本文档及任何相关或随附软件或硬件不做任何明示或暗示的保证，包括但不限于适销性或特定用途适用性的保证。

8. 使用 **Renesas Electronics** 产品时，请参考最新的产品信息（数据手册、用户手册、应用笔记、可靠性手册中的“半导体器件使用一般注意事项”等），并确保使用条件在 **Renesas Electronics** 规定的最大额定值、工作电源电压范围、散热特性等范围内。
9. 尽管 **Renesas Electronics** 努力提升 **Renesas Electronics** 产品的质量和可靠性，但半导体产品具有特定特性，例如在一定概率下发生故障或出现错误操作。客户应采取适当的设计和安全措施来应对这些风险。
10. 有关环境事项（如每个 **Renesas Electronics** 产品的环境兼容性），请联系 **Renesas Electronics** 销售办事处。您有责任仔细调查所有适用法律，确保您的产品或系统符合这些法律。
11. **Renesas Electronics** 产品和技术不得用于或被纳入任何其制造、使用或销售受任何适用的国内外法律或法规禁止的产品或系统。
12. **Renesas Electronics** 产品的购买者或分销商，或任何将产品分销、处置或以其他方式销售或转让给第三方的当事方，有责任将这些注意事项告知接收方。

13. 本文档未经 Renesas Electronics 事先书面同意，不得以任何形式全部或部分重印、复制或翻印。请联系 Renesas Electronics 销售办事处以了解更多信息。

（注 1）本文档中使用的"Renesas Electronics"指 Renesas Electronics Corporation，也包括其直接或间接控制的子公司。

（注 2）"Renesas Electronics 产品"指由 Renesas Electronics 开发或制造的任何产品。

（Rev.5.0-1 2020 年 10 月）

公司总部

TOYOSU FORESIA, 3-2-24 Toyosu, Koto-ku, Tokyo 135-0061, Japan

TOYOSU FORESIA, 3-2-24 Toyosu, Koto-ku,
Tokyo 135-0061, Japan www.renesas.com

Trademarks

Renesas and the Renesas logo are trademarks of Renesas Electronics Corporation. All trademarks and registered trademarks are the property of their respective owners.

联系信息

有关产品、技术、最新版本文档或就近销售办事处的更多信息，请访问：

www.renesas.com/contact/

For further information on a product, technology, the most up-to-date version of a document, or your nearest sales office, please visit:
www.renesas.com/contact/.

© 2026 Renesas Electronics Corporation. 保留所有权利。
© 2026 Renesas Electronics Corporation. All rights reserved